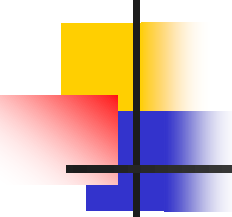


Unit-1 Part-3



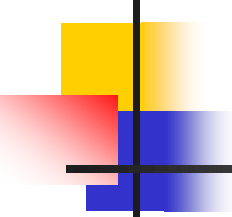
Gate-Level Minimization

- Introduction
- The map method
- Four-variable K-Map
- Product of-Sums Simplification
- Don't –Care Conditions
- NAND and NOR implementation
- Other Two level Implementations



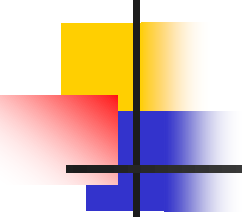
3-1 Introduction

- *Gate-level minimization* refers to the design task of finding an optimal gate-level implementation of Boolean functions describing a digital circuit.



3-2 The Map Method

- The **complexity of the digital logic gates** that implement a Boolean function is directly related to the **complexity of the algebraic expression** from which the function is implemented.
- Logic minimization
 - Algebraic approaches
 - lack specific rules
 - It is a time consuming process.
 - Re-write the simplified expressions after each step.
 - **The Karnaugh map (K-map)**
 - It is a graphical method, which consists of 2^n cells for 'n' variables.
 - The adjacent cells are differed only in single bit position.
 - If there are more than 5 variables, it is still possible to use Karnaugh maps, but it becomes more difficult to see patterns.
 - Quine-McClukey tabular method
 - It is a tabular method based on the concept of **prime implicants**.
 - **Prime implicant** is a product *or sum* term, which can't be further reduced by combining with any other product *or sum* terms of the given Boolean function.



Karnaugh map (K-map)

- A simple straight forward procedure.
- A pictorial form of a truth table.
- Applicable if the # of variables < 5
- However, as the number of variables increases it becomes more difficult to see patterns, and computer methods start to become more attractive.
- A diagram made up of squares
 - Each square represents one minterm.
- Boolean function
 - Sum of minterms.
 - Sum of products (or product of sum) in the simplest form.
 - A minimum number of terms.
 - A minimum number of literals.
 - The simplified expression may not be unique.

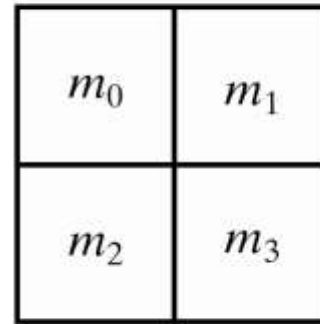
Two-Variable Map

A two-variable map

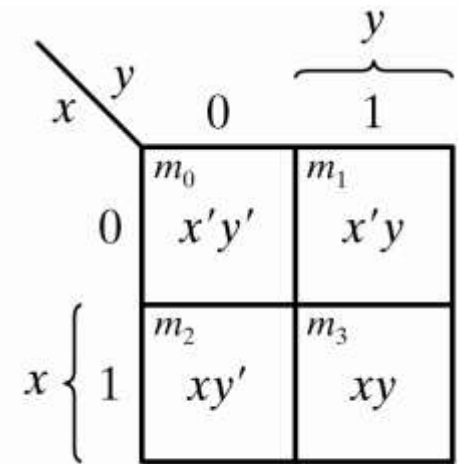
- four minterms
- $x' = \text{row } 0$; $x = \text{row } 1$
- $y' = \text{column } 0$;
 $y = \text{column } 1$

a truth table in square diagram

- xy
- $x+y = x'y + xy' + xy$

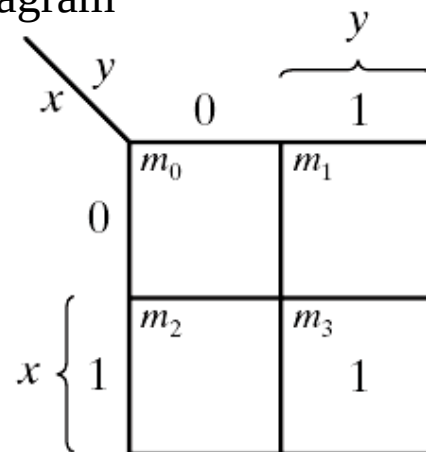


(a)

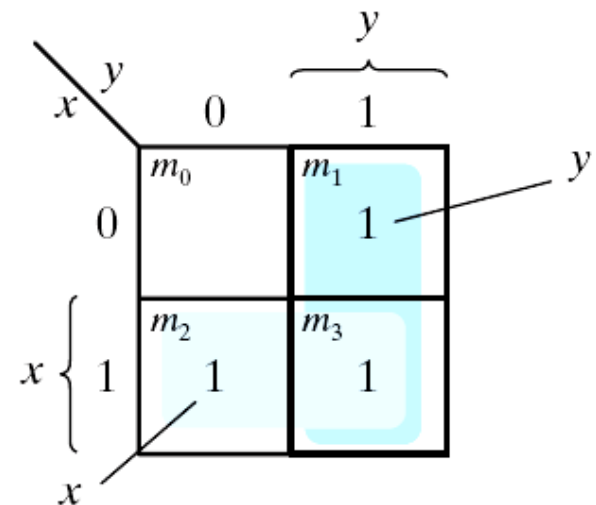


(b)

Fig. 3-1 Two-variable Map



(a) xy



(b) $x + y$

Fig. 3.2 Representation of functions in the map

A three-variable map

- Eight minterms
- The Gray code sequence
 - 00, 01, 10, 11 (binary seq.) = 00, 01, 11, 10 (gray code seq.)
- Any two adjacent squares in the map differ by only on variable
 - Primed in one square and unprimed in the other
 - e.g., m_5 and m_7 can be simplified
 - $m_5 + m_7 = xy'z + xyz = xz (y' + y) = xz$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		y				
		$\overbrace{\hspace{1.5cm}}$				
		xz	00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$	
x	1	$xy'z'$	$xy'z$	xyz	xyz'	
		$\underbrace{\hspace{1.5cm}}$				z

Fig. 3-3 Three-variable Map (a)

(b)

- Example 3-1
 - $F(x,y,z) = \Sigma(2,3,4,5)$
 - $F = x'y + xy'$

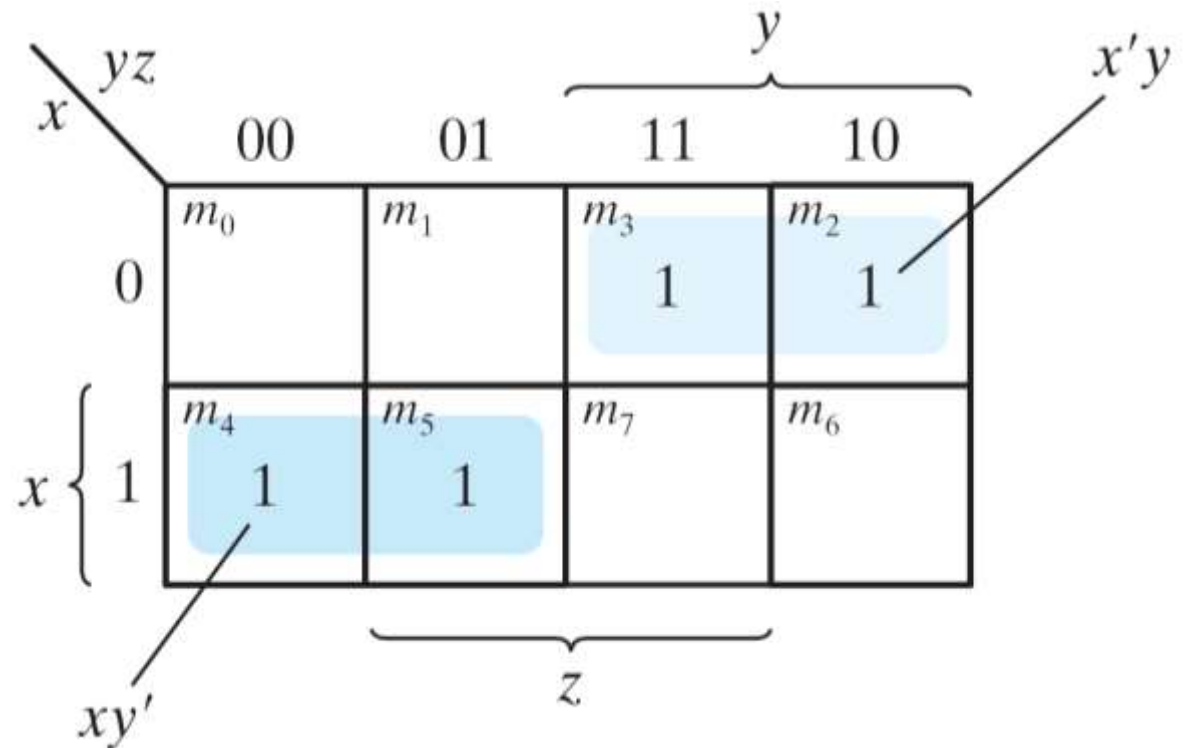


Fig. 3-4 Map for Example 3-1; $F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$

- m_0 and m_2 (m_4 and m_6) are adjacent
- $m_0 + m_2 = x'y'z' + x'yz' = x'z' (y' + y) = x'z'$
- $m_4 + m_6 = xy'z' + xyz' = xz' (y' + y) = xz'$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

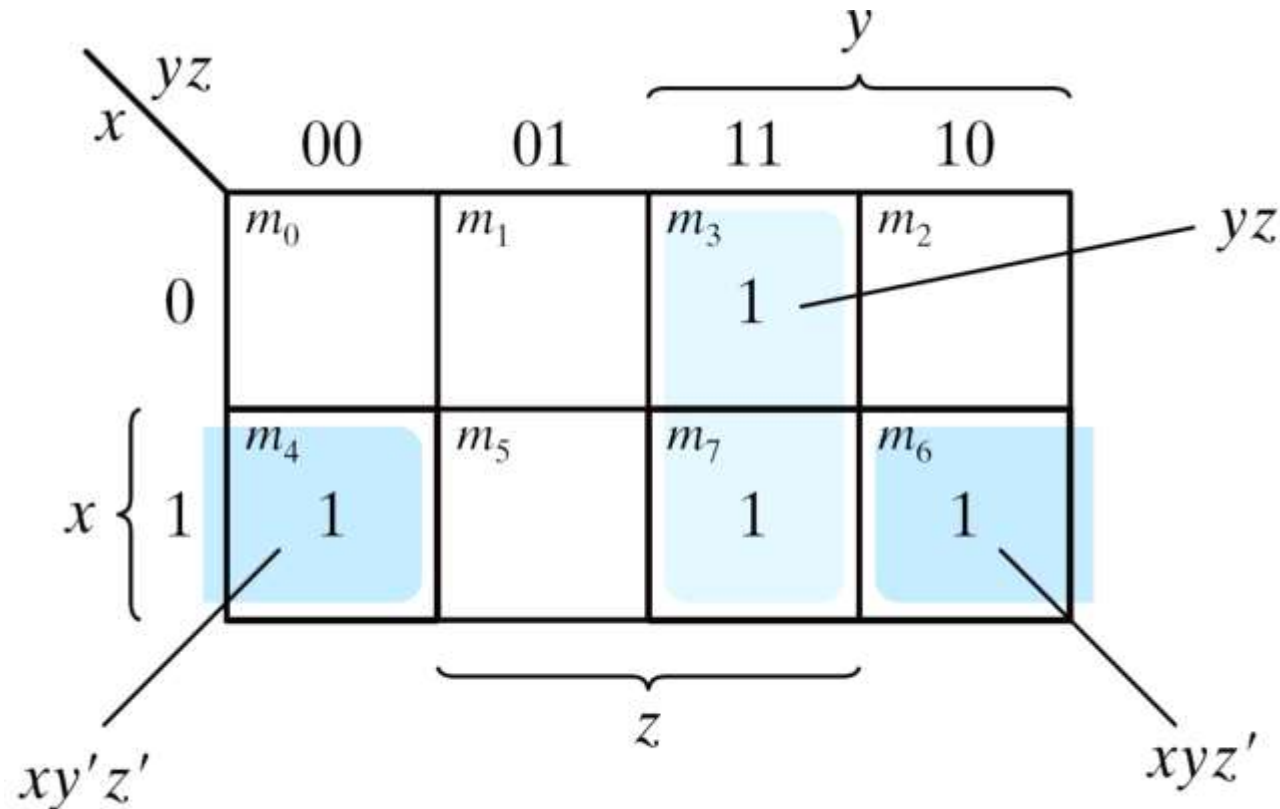
		y			
		xz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

(b)

Fig. 3-3 Three-variable Map

- Example 3-2

- $F(x,y,z) = \Sigma(3,4,6,7) = yz + xz'$



Note: $xy'z' + xyz' = xz'$

Fig. 3-5 Map for Example 3-2; $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$ Digital Circuits 3-9

■ Four adjacent squares

- 2, 4, 8 and 16 squares

- $m_0 + m_2 + m_4 + m_6 = x'y'z' + x'yz' + xy'z' + xyz'$
 $= x'z'(y' + y) + xz'(y' + y)$
 $= x'z' + xz' = z'$

- $m_1 + m_3 + m_5 + m_7 = x'y'z + x'yz + xy'z + xyz$
 $= x'z(y' + y) + xz(y' + y)$
 $= x'z + xz = z$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		y			
		xz			
		00	01	11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

Fig. 3-3 Three-variable Map

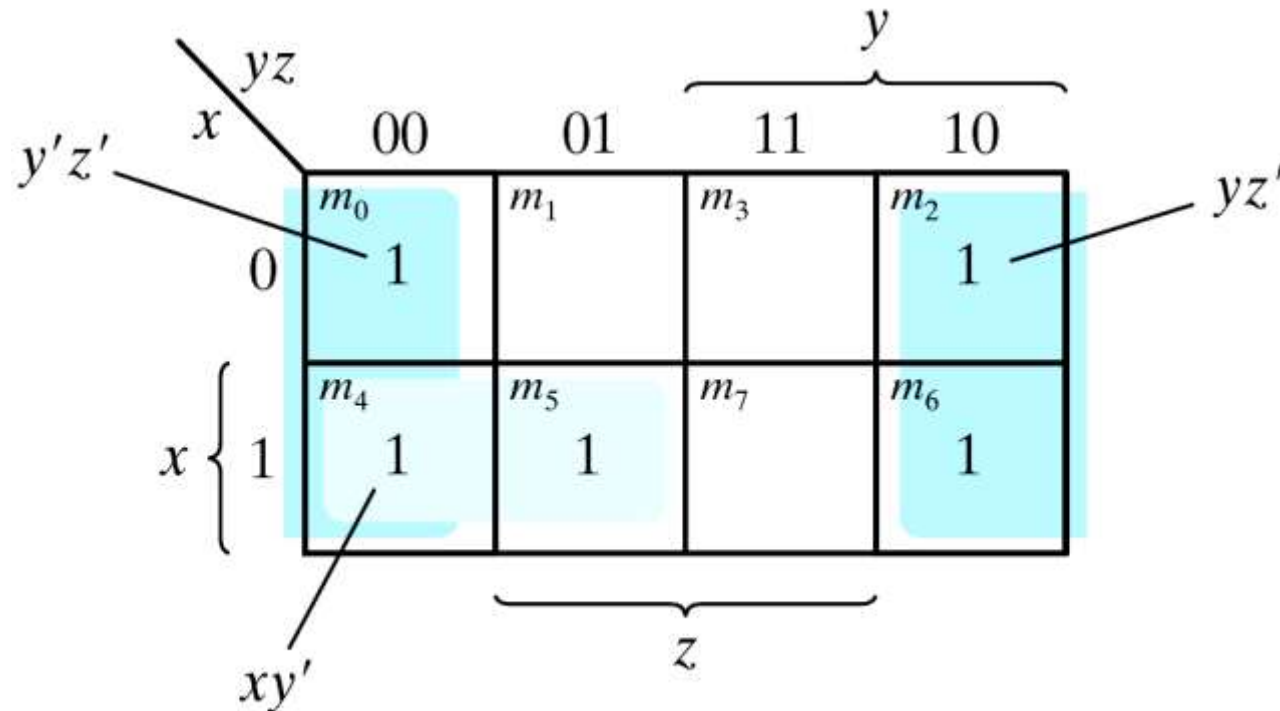
(a)

(b)

■ Example 3-3

■ $F(x,y,z) = \Sigma(0,2,4,5,6)$

■ $F = z' + xy'$



Note: $y'z' + yz' = z'$

Fig. 3-6 Map for Example 3-3; $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

■ Example 3-4

- $F = A'C + A'B + AB'C + BC$
- express it in sum of minterms
- find the minimal sum of products expression

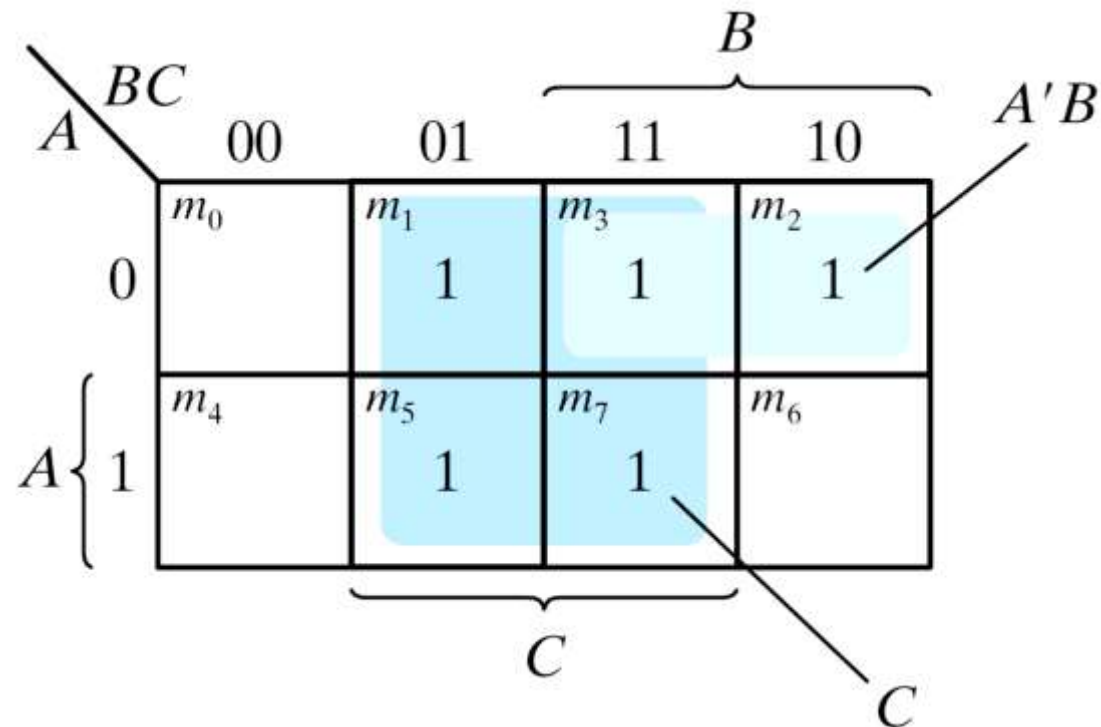
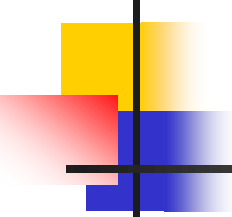


Fig. 3-7 Map for Example 3-4; $A'C + A'B + AB'C + BC = C + A'B$



Worksheet-1

3.1* Simplify the following Boolean functions, using three-variable maps:

(a) $F(x, y, z) = \Sigma(0, 2, 4, 5)$

(b) $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$

(c) $F(x, y, z) = \Sigma(0, 1, 2, 3, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 7)$

3.2 Simplify the following Boolean functions, using three-variable maps:

(a)* $F(x, y, z) = \Sigma(0, 1, 5, 7)$

(b)* $F(x, y, z) = \Sigma(1, 2, 3, 6, 7)$

(c) $F(x, y, z) = \Sigma(2, 3, 4, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$

(e) $F(x, y, z) = \Sigma(0, 2, 4, 6)$

(f) $F(x, y, z) = \Sigma(3, 4, 5, 6, 7)$

3.3* Simplify the following Boolean expressions, using three-variable maps:

(a)* $xy + x'y'z' + x'yz'$

(b)* $x'y' + yz + x'yz'$

(c)* $F(x, y, z) = x'y + yz' + y'z'$

(d) $F(x, y, z) = x'yz + xy'z' + xy'z$

3-3 Four-Variable Map

■ The map

- 16 minterms
- combinations of 2, 4, 8, and 16 adjacent squares

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

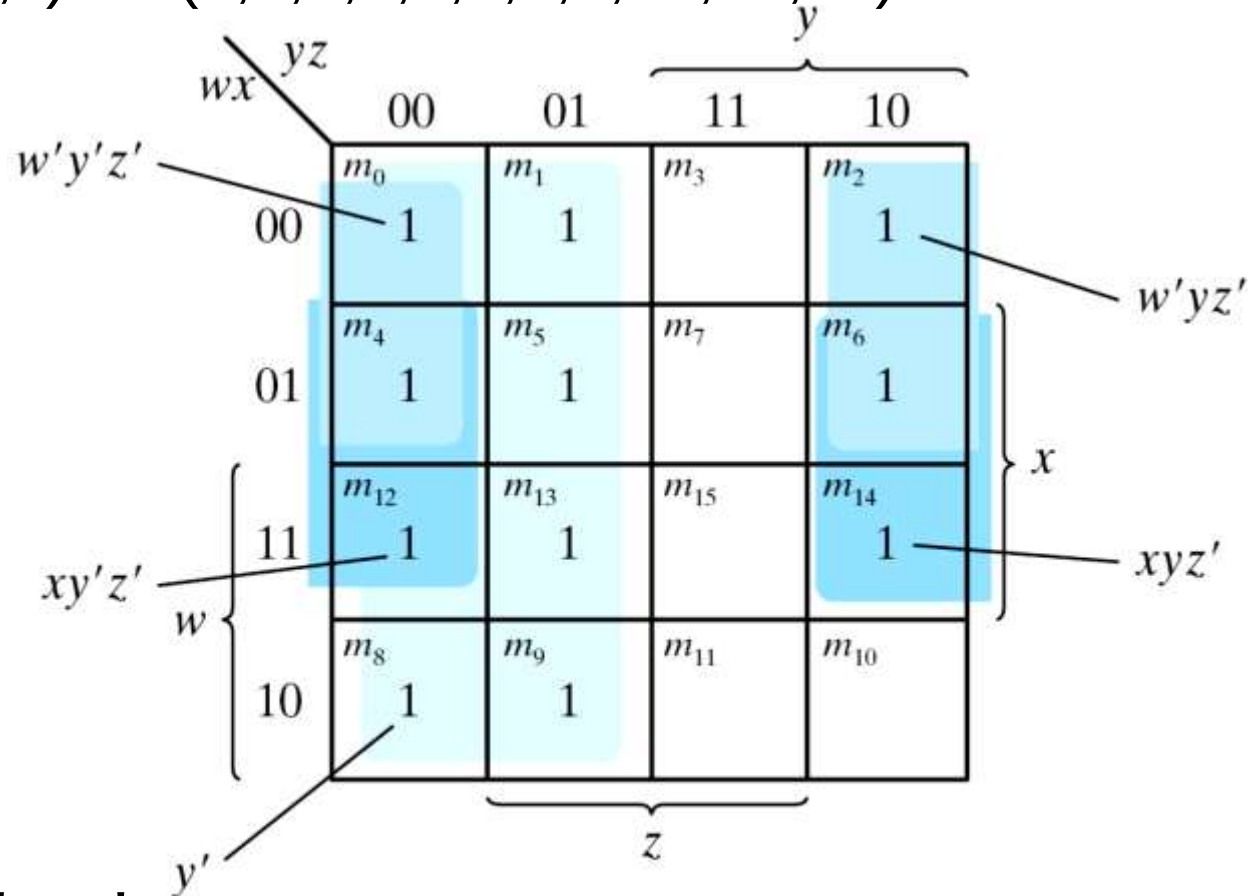
		y			
		yz		11	10
w	wx	00	01	11	10
	00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$
	01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$
	11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$
	10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$
		z			

(b)

Fig. 3-8 Four-variable Map

Example 3-5

■ $F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$



■ $F = y' + w'z' + xz'$

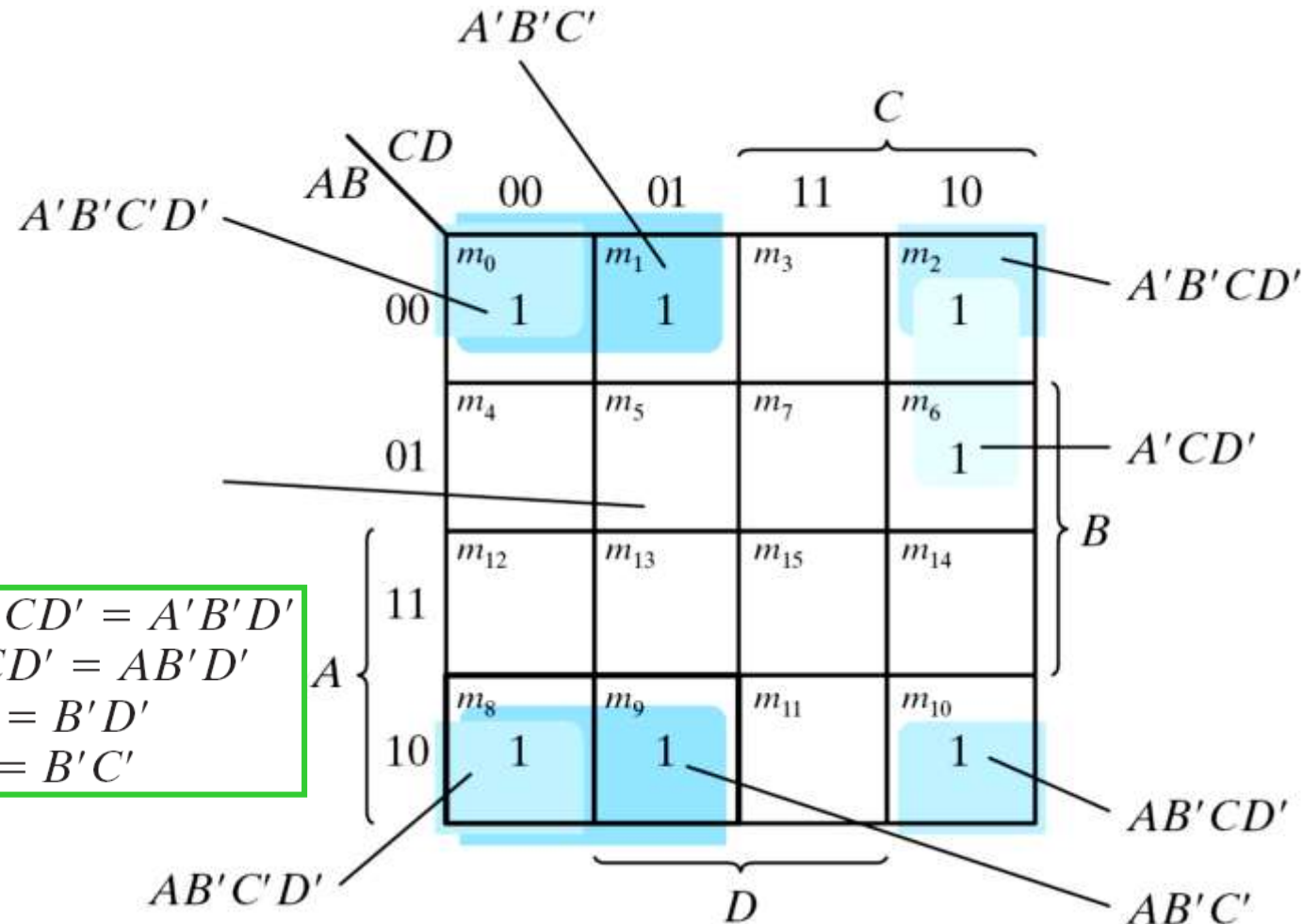
Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

Fig. 3-9 Map for Example 3-5; $F(w, x, y, z)$

$= \Sigma (0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$

■ Example 3-6 Simplify the Boolean function

$$F = A'B'C' + B'CD' + A'B'C'D' + AB'C'$$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

Fig.3-10 Map for Example 3-6; $A'B'C' + B'CD' + A'B'CD' + AB'C' = B'D' + B'C' + A'CD'$

Worksheet -2

3.4 Simplify the following Boolean functions, using *Karnaugh* maps:

(a)* $F(x, y, z) = \Sigma(2, 3, 6, 7)$

(b)* $F(A, B, C, D) = \Sigma(4, 6, 7, 15)$

(c)* $F(A, B, C, D) = \Sigma(3, 7, 11, 13, 14, 15)$

(d)* $F(w, x, y, z) = \Sigma(2, 3, 12, 13, 14, 15)$

(e) $F(w, x, y, z) = \Sigma(11, 12, 13, 14, 15)$

(f) $F(w, x, y, z) = \Sigma(8, 10, 12, 13, 14)$

3.5 Simplify the following Boolean functions, using four-variable maps:

(a)* $F(w, x, y, z) = \Sigma(1, 4, 5, 6, 12, 14, 15)$

(b) $F(A, B, C, D) = \Sigma(2, 3, 6, 7, 12, 13, 14)$

(c) $F(w, x, y, z) = \Sigma(1, 3, 4, 5, 6, 7, 9, 11, 13, 15)$

(d)* $F(A, B, C, D) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$

3.6 Simplify the following Boolean expressions, using four-variable maps:

(a)* $A'B'C'D' + AC'D' + B'CD' + A'BCD + BC'D$

(b)* $x'z + w'xy' + w(x'y + xy')$

(c) $A'B'C'D + AB'D + A'BC' + ABCD + AB'C$

(d) $A'B'C'D' + BC'D + A'C'D + A'BCD + ACD'$

3.7 Simplify the following Boolean expressions, using four-variable maps:

(a)* $w'z + xz + x'y + wx'z$

(b) $AD' + B'C'D + BCD' + BC'D$

(c)* $AB'C + B'C'D' + BCD + ACD' + A'B'C + A'BC'D$

(d) $wxy + xz + wx'z + w'x$

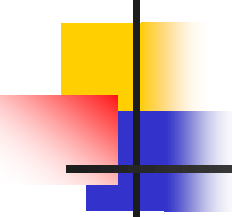
3.8 Find the minterms of the following Boolean expressions by first plotting each function in a map:

(a)* $xy + yz + xy'z$

(b)* $C'D + ABC' + ABD' + A'B'D$

(c) $wyz + w'x' + wxz'$

(d) $A'B + A'CD + B'CD + BC'D'$

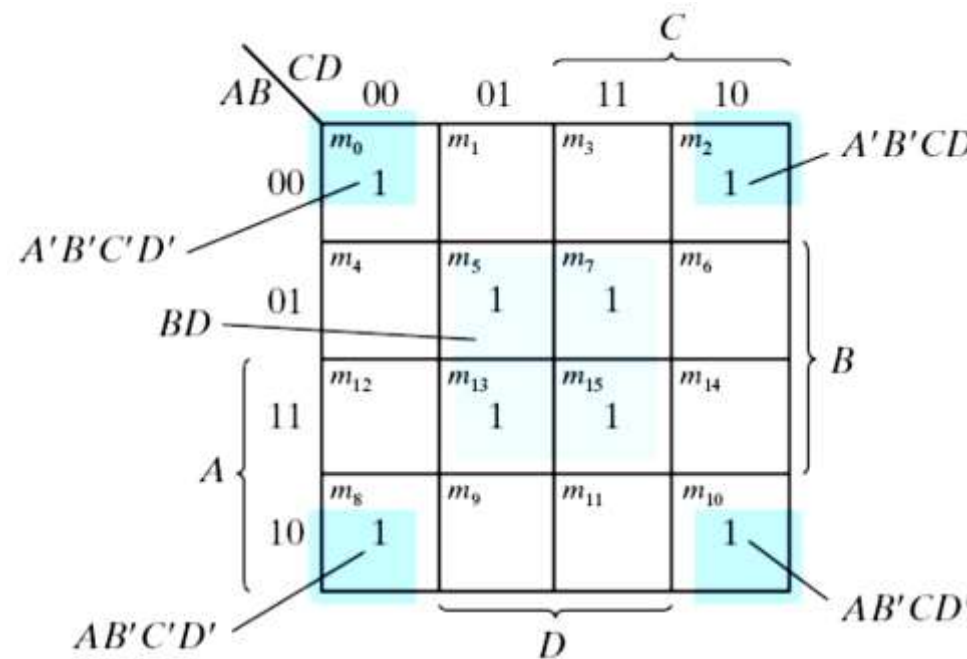


■ Prime Implicants

- all the minterms are covered.
- minimize the number of terms.
- a prime implicant: a product term obtained by combining the maximum possible number of adjacent squares (combining all possible maximum numbers of squares).
- **essential**: a minterm is covered by only one prime implicant, that prime implicant is called essential.

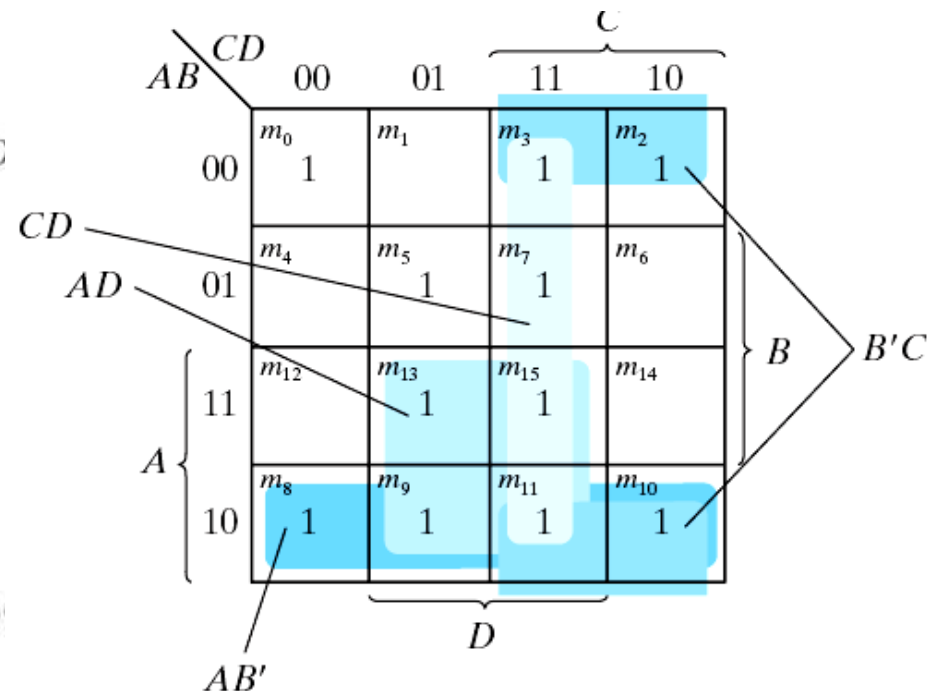
Consider $F(A,B,C,D) = \sum(0,2,3,5,7,8,9,10,11,13,15)$

- the simplified expression may not be unique
- $F = BD + B'D' + CD + AD = BD + B'D' + CD + AB'$
 $= BD + B'D' + B'C + AD = BD + B'D' + B'C + AB'$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$

(a) Essential prime implicants
 BD and $B'D'$



(b) Prime implicants CD , $B'C$,
 AD , and AB'

Fig. 3-11 Simplification Using Prime Implicants

3-4 Five-Variable Map

- Map for more than four variables becomes complicated
 - five-variable map: two four-variable map (one on the top of the other)

$A = 0$

		DE		D	
		00	01	11	10
BC	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

E

$A = 1$

		DE		D	
		00	01	11	10
BC	00	16	17	19	18
	01	20	21	23	22
	11	28	29	31	30
	10	24	25	27	26

E

Fig. 3-12 Five-variable Map

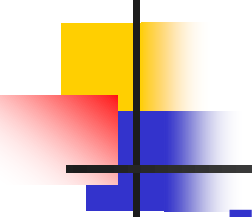
- 
- Table 3.1 shows the relationship between the number of adjacent squares and the number of literals in the term.

Table 3.1

The Relationship between the Number of Adjacent Squares and the Number of Literals in the Term

Number of Adjacent Squares		Number of Literals in a Term in an n -variable Map			
K	2^k	$n = 2$	$n = 3$	$n = 4$	$n = 5$
0	1	2	3	4	5
1	2	1	2	3	4
2	4	0	1	2	3
3	8		0	1	2
4	16			0	1
5	32				0

■ Example 3-7

■ $F = \Sigma(0,2,4,6,9,13,21,23,25,29,31)$

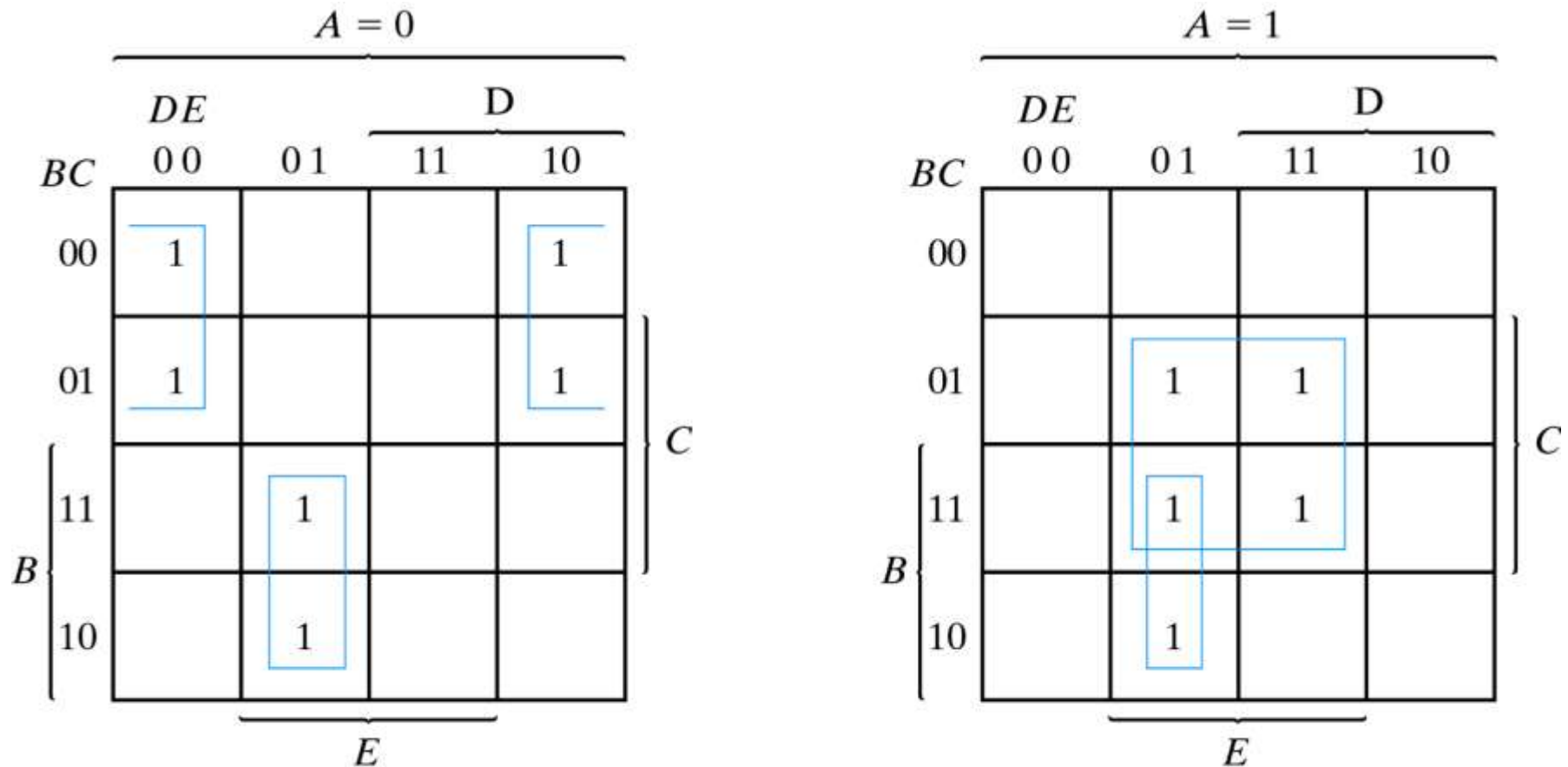
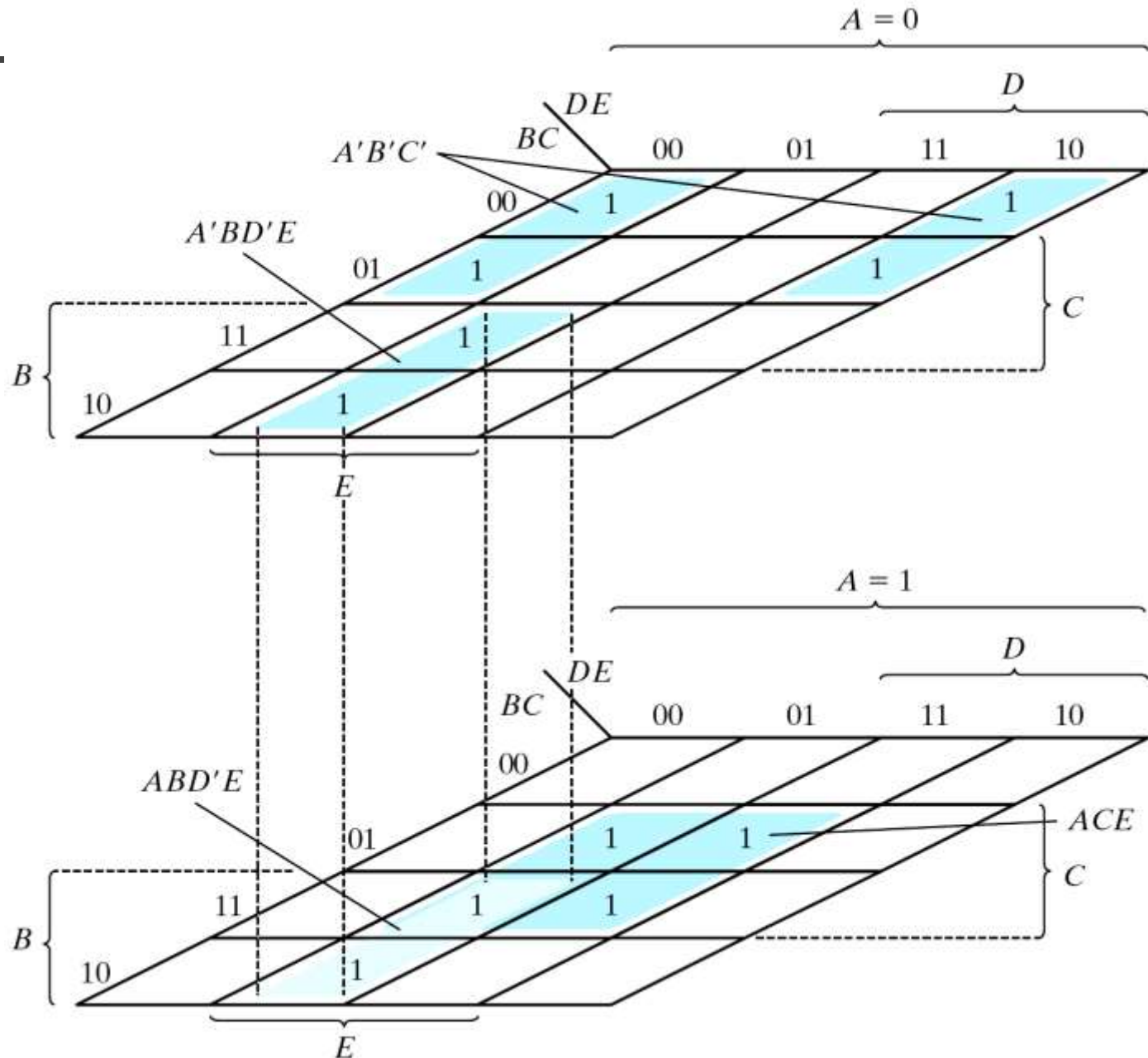


Fig. 3-13 Map for Example 3-7; $F = A'B'E' + BD'E + ACE$



$$F = A'B'E' + BD'E + ACE$$

♣ Another Map for Example 3-7



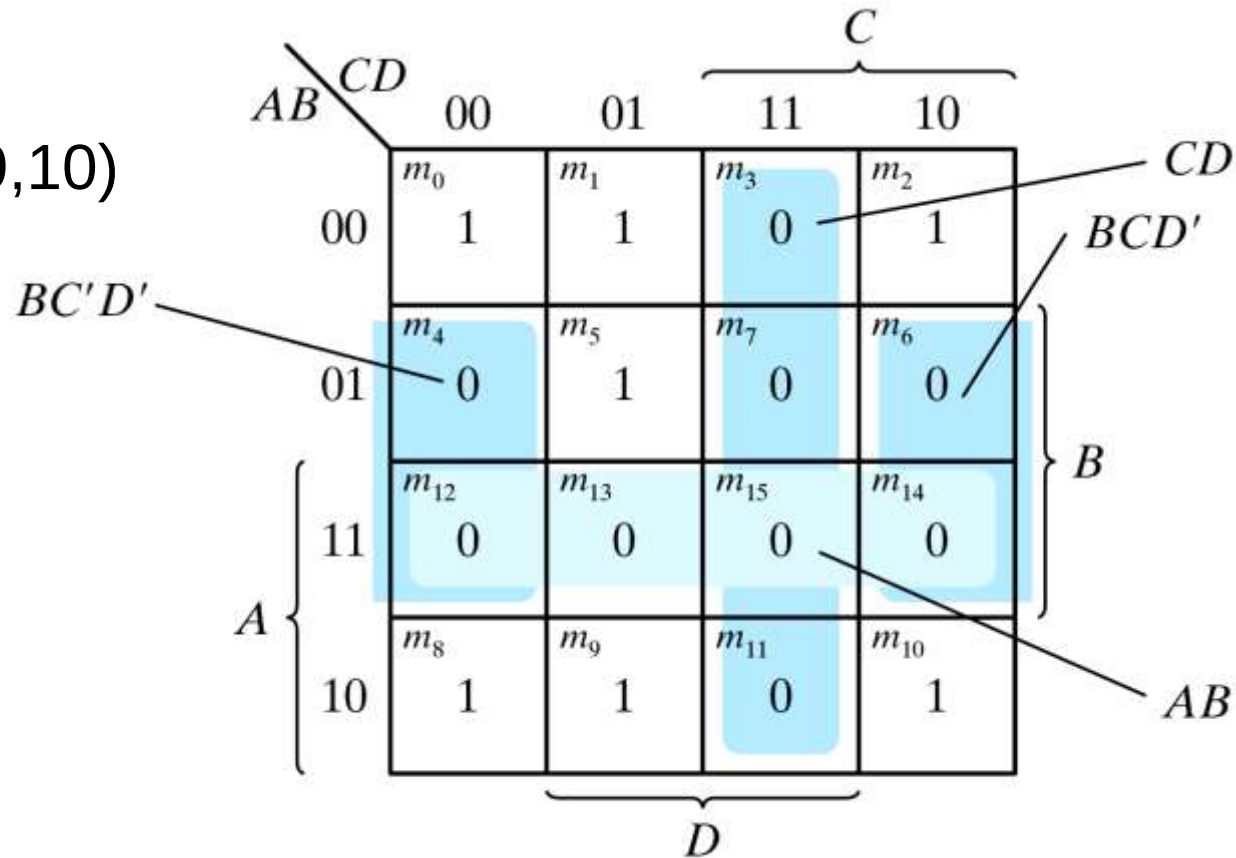
3-5 Product of Sums Simplification

- Approach #1
 - Simplified F' in the form of sum of products
 - Apply DeMorgan's theorem $F = (F')'$
 - F' : sum of products $\Rightarrow F$: product of sums
- Approach #2: duality
 - combinations of maxterms (like it was minterms)
 - $M_0 M_1 = (A+B+C+D)(A+B+C+D')$
 $= (A+B+C)+(DD')$
 $= A+B+C$

	CD			
AB	00	01	11	10
00	M_0	M_1	M_3	M_2
01	M_4	M_5	M_7	M_6
11	M_{12}	M_{13}	M_{15}	M_{14}
10	M_8	M_9	M_{11}	M_{10}

■ Example 3-8

■ $F = \Sigma(0,1,2,5,8,9,10)$



Note: $BC'D' + BCD' = BD'$

- $F' = AB + CD + BD'$
- Apply DeMorgan's theorem; $F = (A' + B')(C' + D')(B' + D)$
- Or think in terms of maxterms

- Gate implementation of the function of Example 3-8

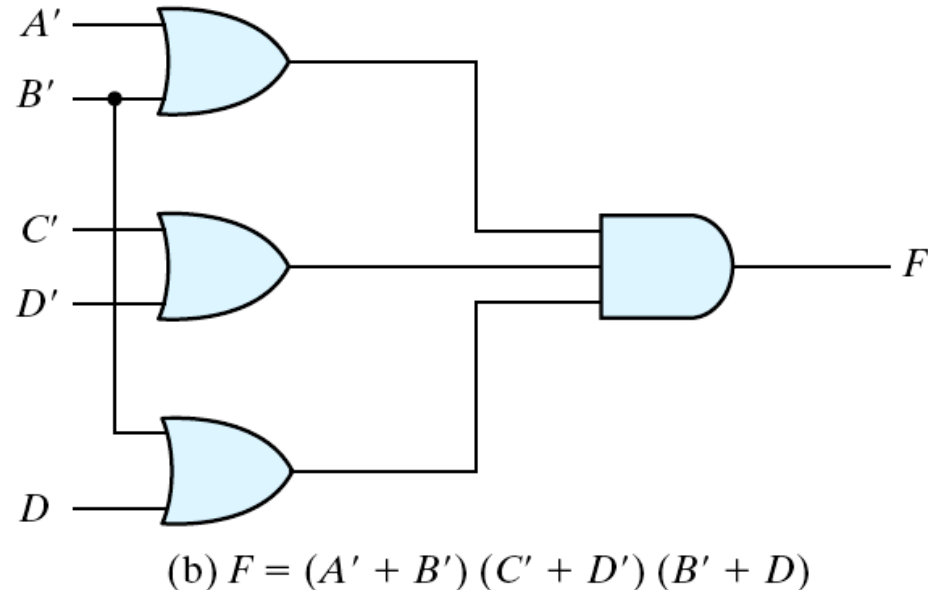
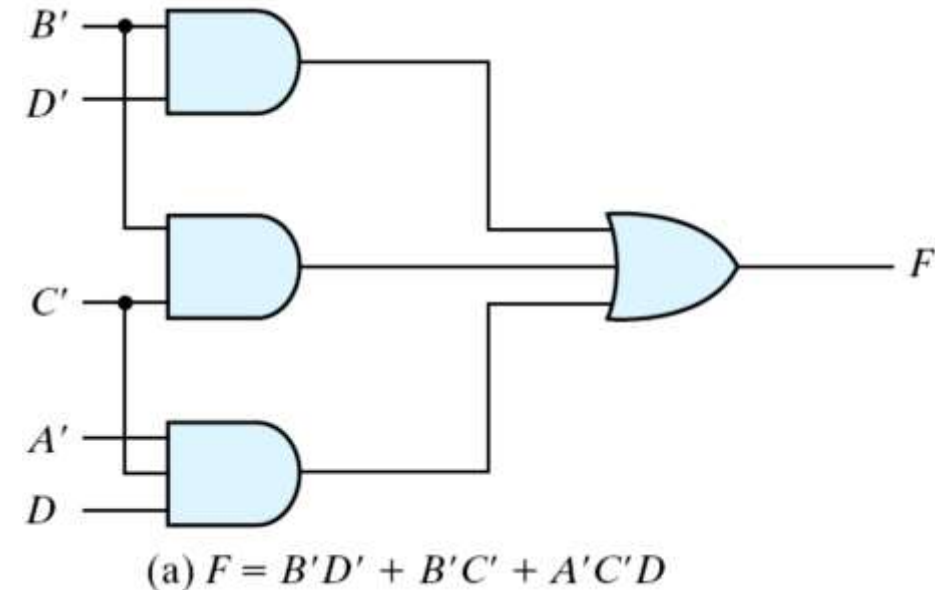
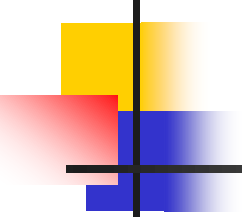


Fig. 3-15 Gate Implementation of the Function of Example 3-8

- 
- Consider the function defined in Table 3.2.

In sum-of-minterm:

$$F(x, y, z) = \sum(1, 3, 4, 6)$$

In product-of-maxterm:

$$F(x, y, z) = \prod(0, 2, 5, 7)$$

Table 3.2

Truth Table of Function F

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

- Consider the function defined in Table 3.2.

Combine the 1's:

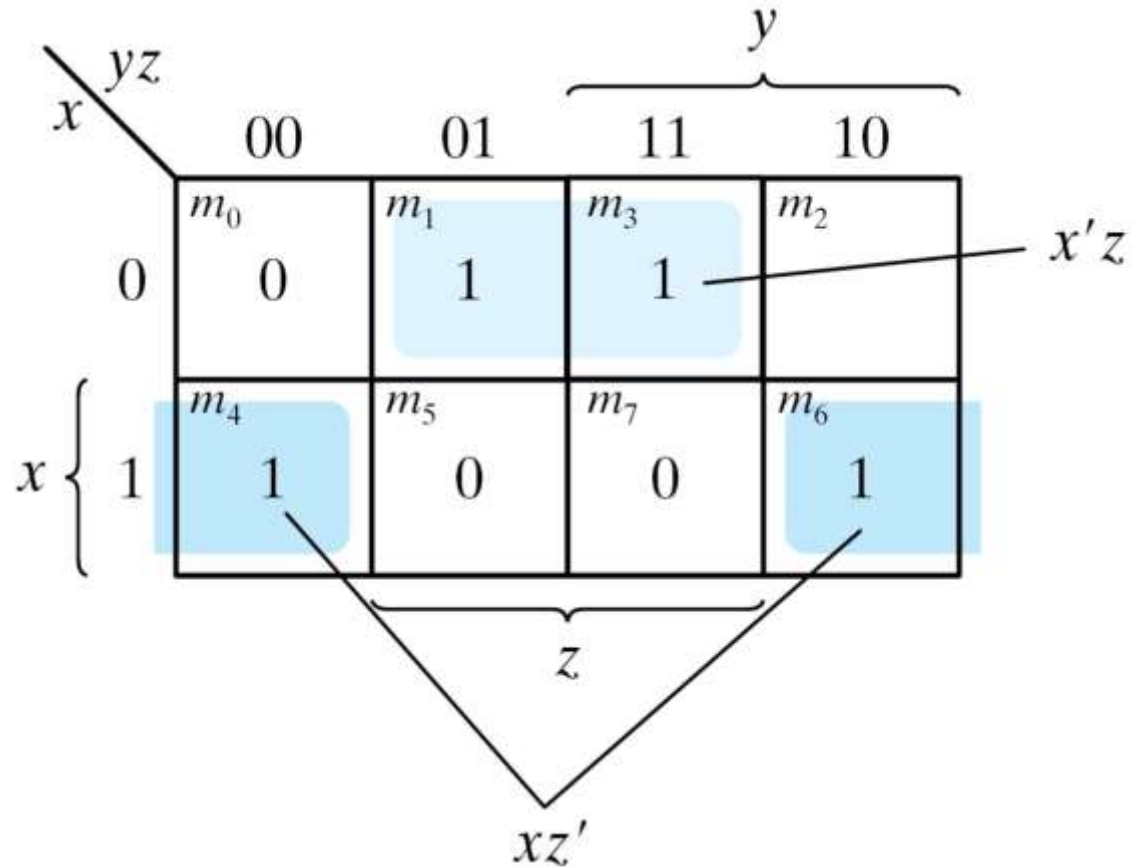
$$F = x'z + xz'$$

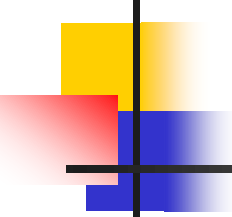
Combine the 0's :

$$F' = xz + x'z'$$

Taking the complement of F'

$$F(x, y, z) = (x' + z')(x + z)$$

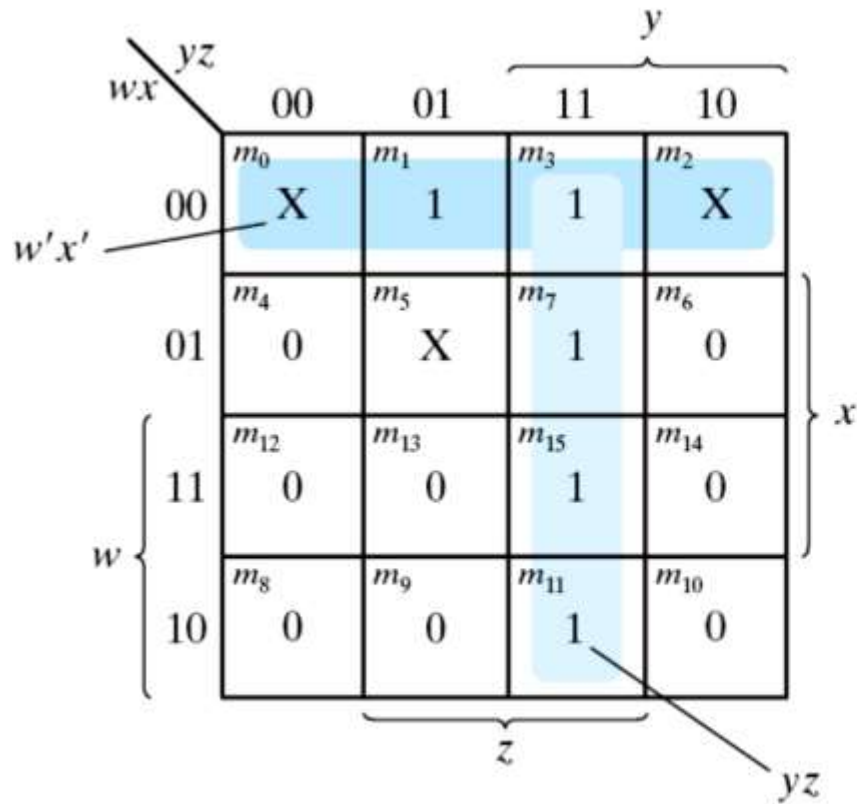




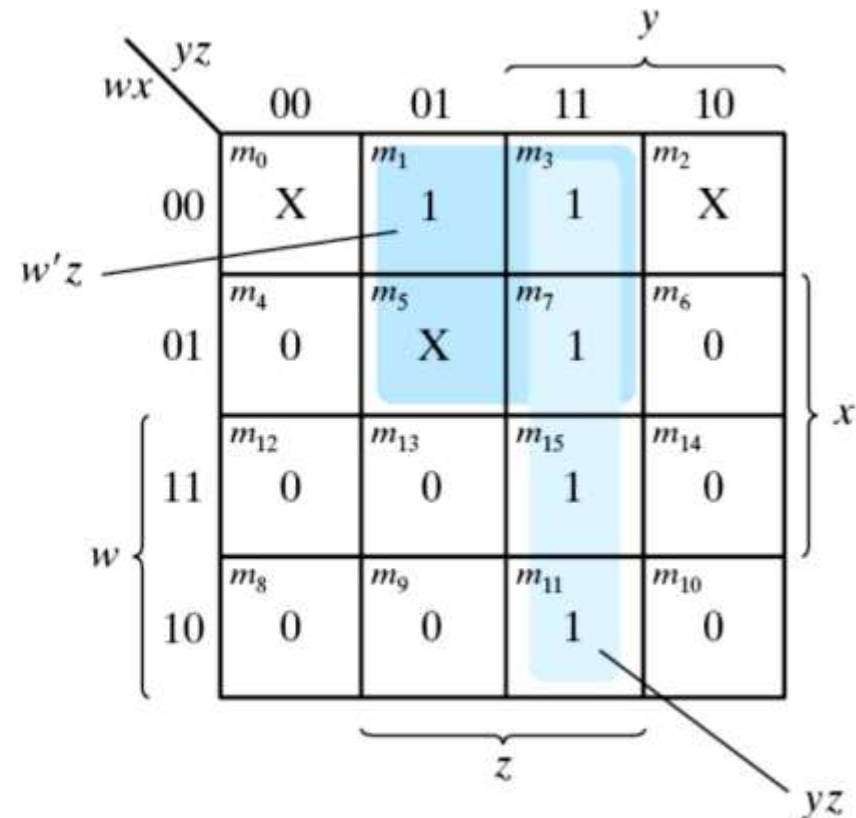
3-6 Don't-Care Conditions

- The value of a function is not specified for certain combinations of variables
 - BCD; 1010-1111: don't care
- The don't care conditions can be utilized in logic minimization
 - can be implemented as 0 or 1
- Example 3-9
 - $F(w,x,y,z) = \Sigma(1,3,7,11,15)$
 - $d(w,x,y,z) = \Sigma(0,2,5)$

- $F = yz + w'x'$; $F = yz + w'z$
- $F = \Sigma(0,1,2,3,7,11,15)$; $F = \Sigma(1,3,5,7,11,15)$
- either expression is acceptable



(a) $F = yz + w'x'$



(b) $F = yz + w'z$

Fig. 3-17 Example with don't-care Conditions

- Also apply to products of sum

3-7 NAND and NOR Implementation

- NAND gate is a universal gate
 - can implement any digital system

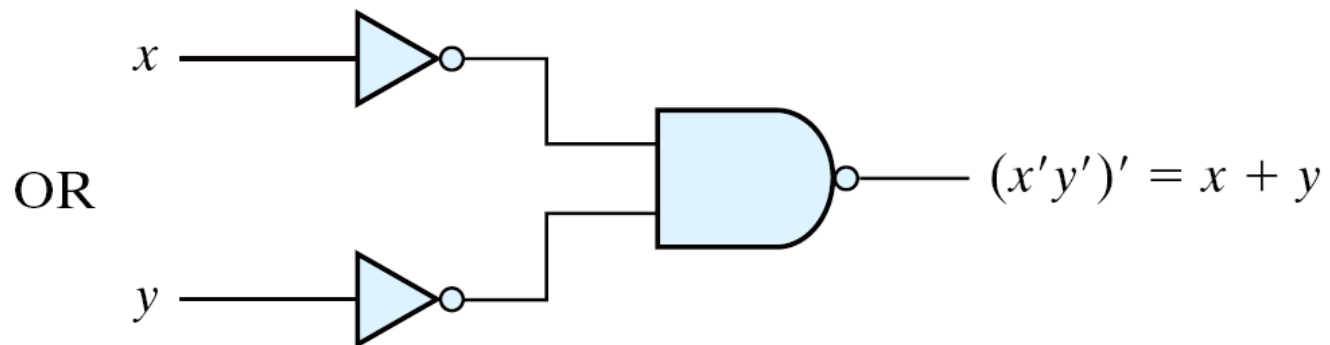
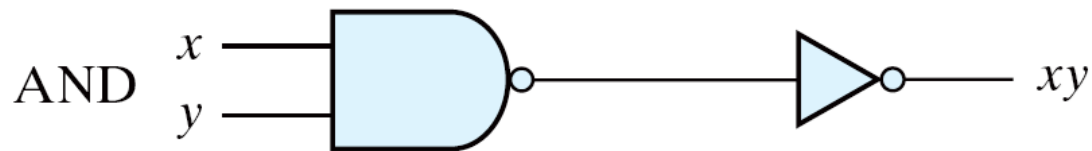
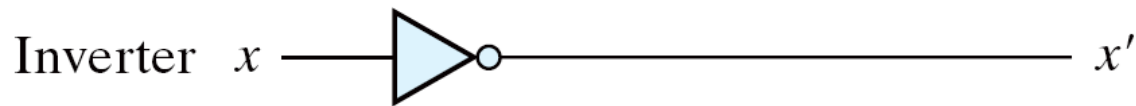


Fig. 3-18 Logic Operations with NAND Gates

- Two graphic symbols for a NAND gate

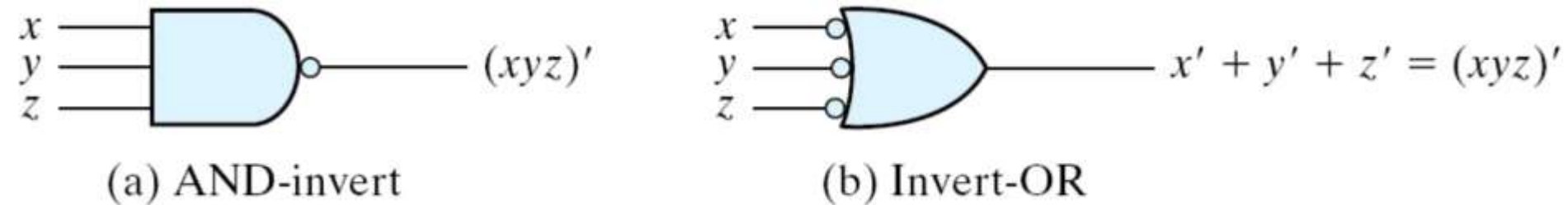


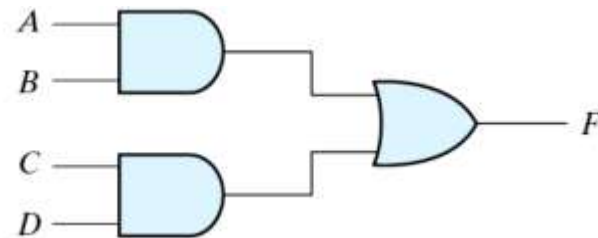
Fig. 3-19 Two Graphic Symbols for NAND Gate

Two-level Implementation

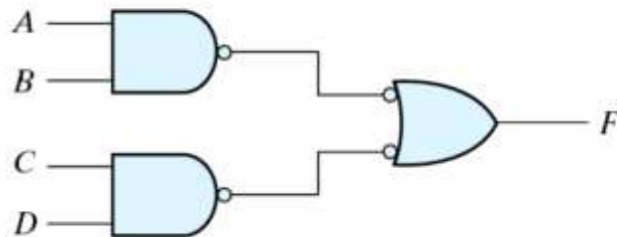
- two-level logic
- NAND-NAND = sum of products
- Example: $F = AB + CD$
- $F = ((AB)' (CD)')' = AB + CD$

Fig. 3-20

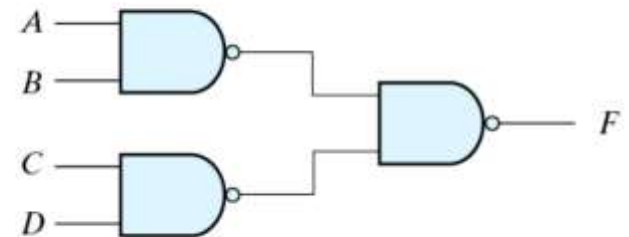
Three ways to implement
 $F = AB + CD$



(a)



(b)



(c)

■ Example 3-10

$$F(x, y, z) = \sum(1, 2, 3, 4, 5, 7) \quad \Rightarrow \quad F(x, y, z) = xy' + x'y + z$$

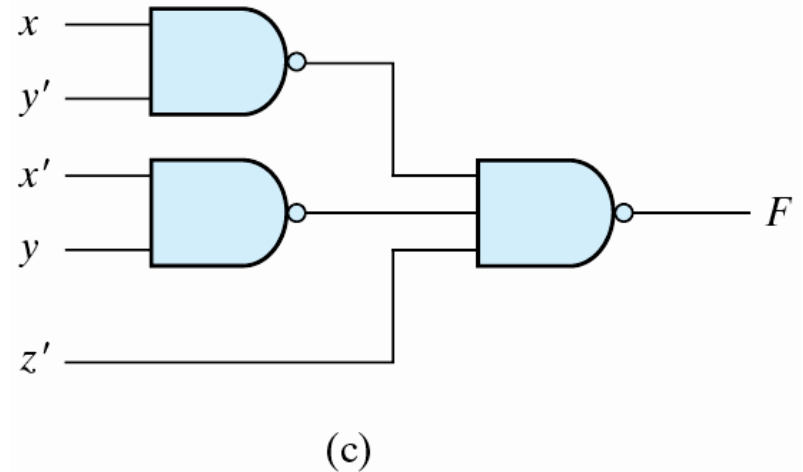
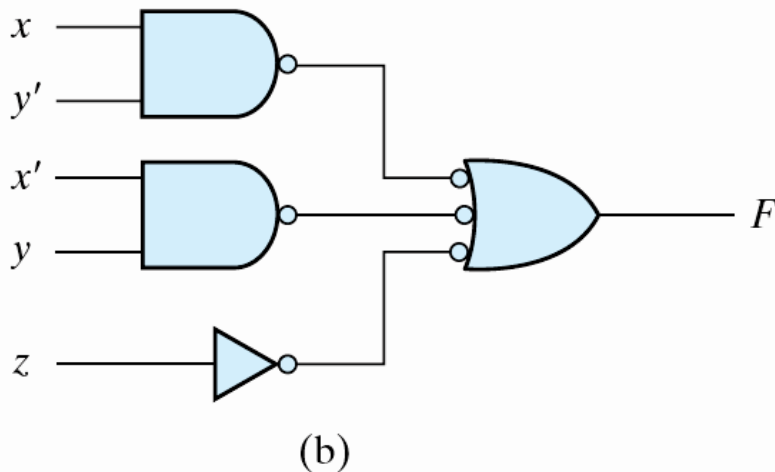
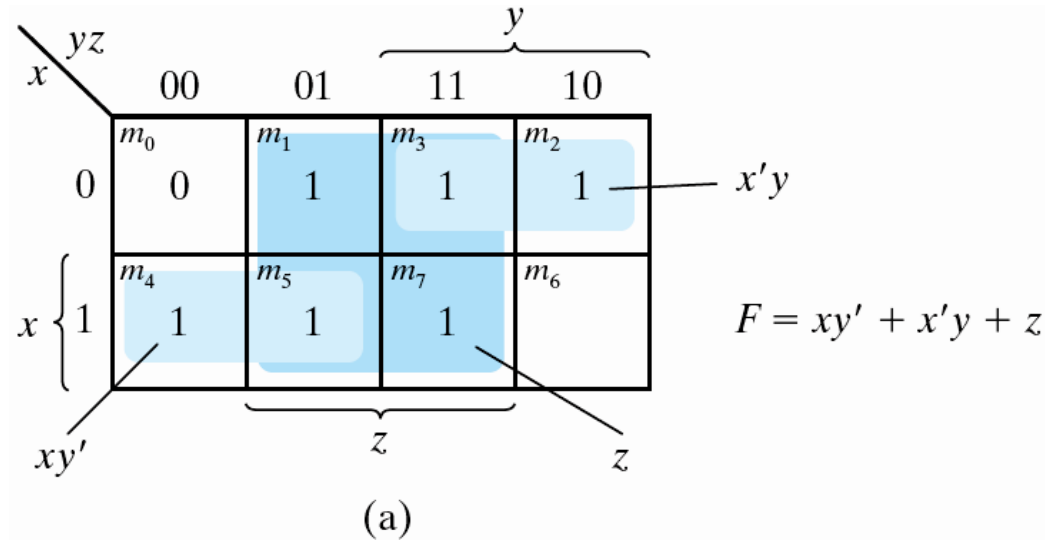


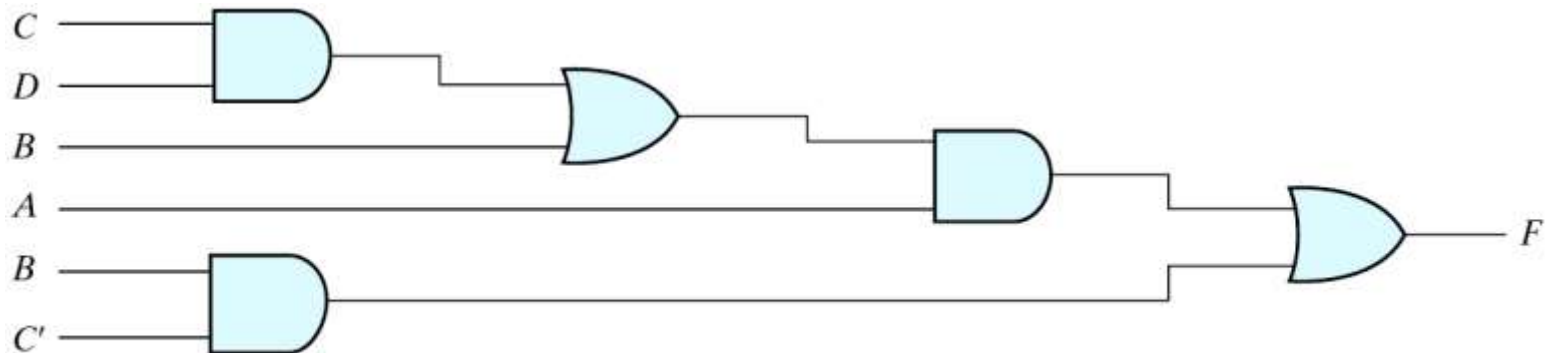
Fig. 3-21 Solution to Example 3-10

- The procedure

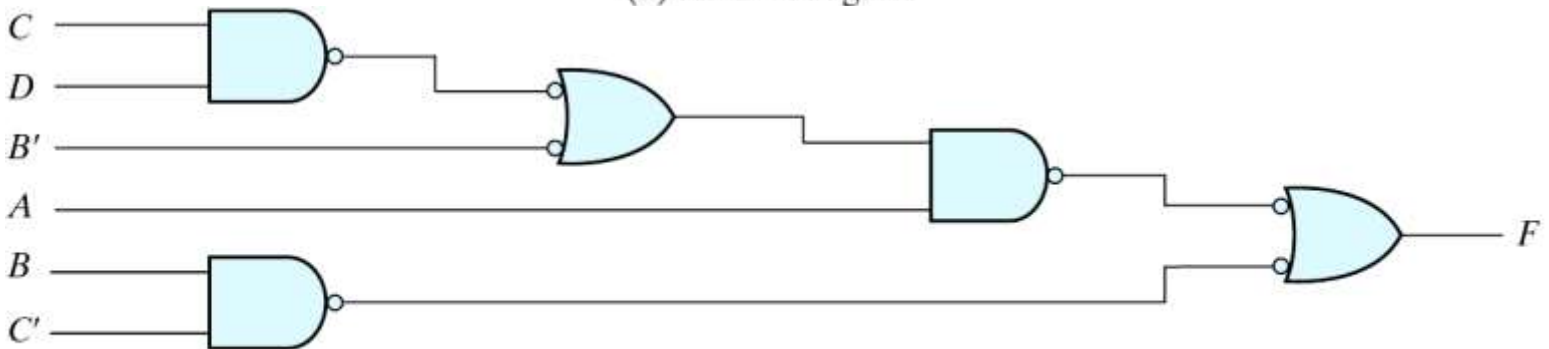
- simplified in the form of sum of products
- a NAND gate for each product term; the inputs to each NAND gate are the literals of the term
- a single NAND gate for the second sum term

Multilevel NAND Circuits

- Boolean function implementation
 - AND-OR logic $\Rightarrow$ NAND-NAND logic
 - AND $\Rightarrow$ NAND + inverter
 - OR: inverter + OR = NAND



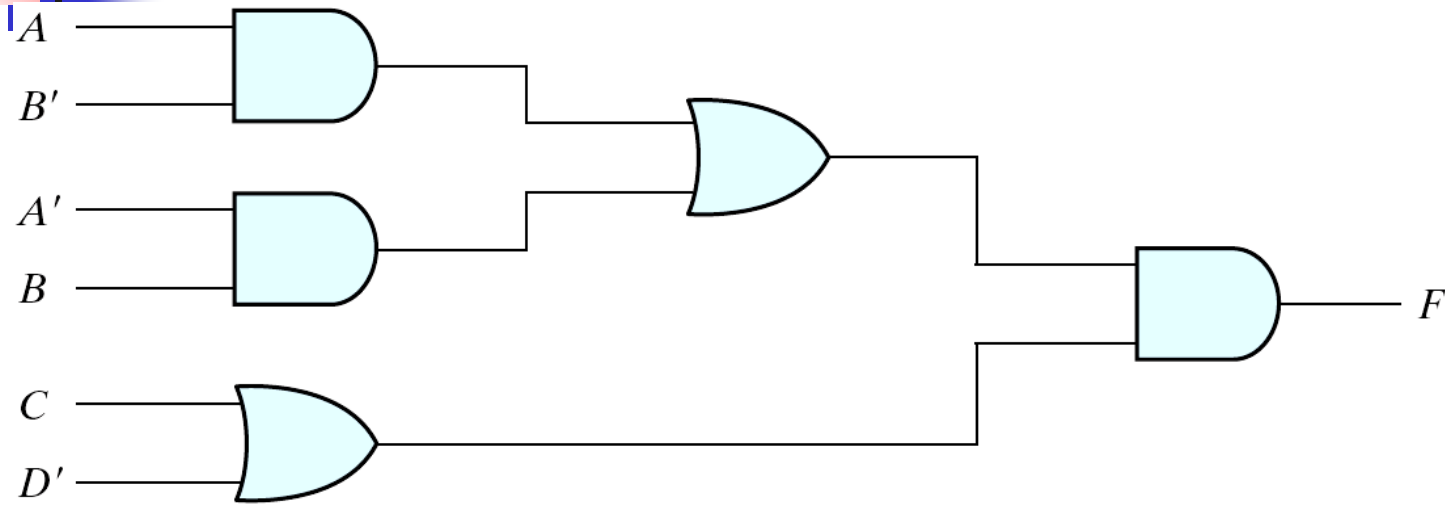
(a) AND-OR gates



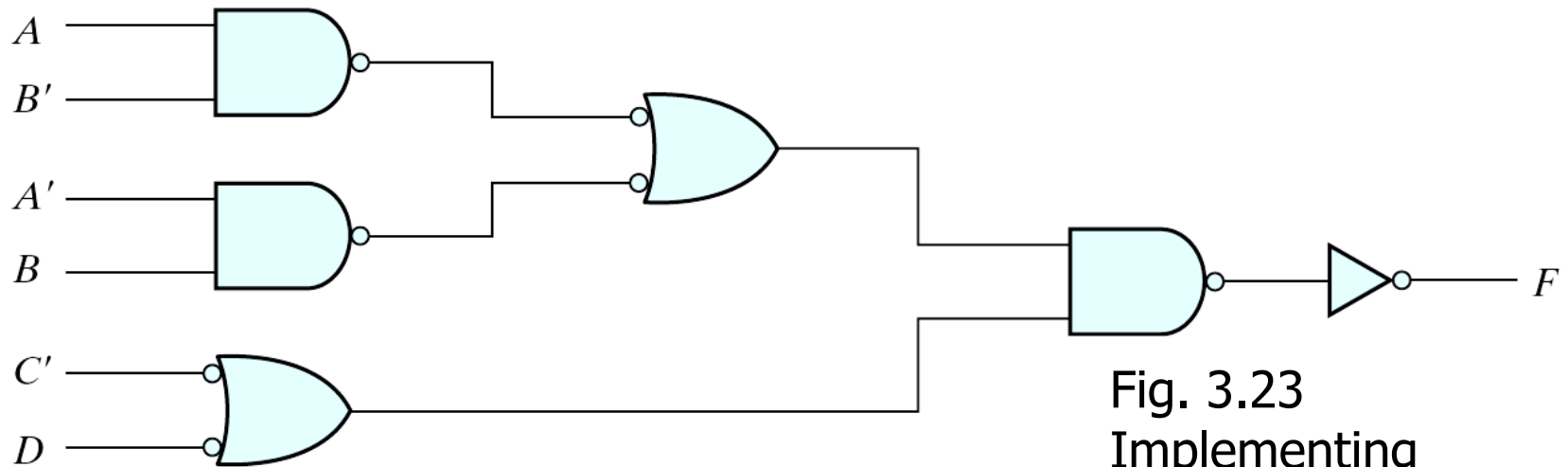
(b) NAND gates

Fig. 3.22
Implementing $F = A(CD + B) + BC'$

NAND Implementation



(a) AND-OR gates



(b) NAND gates

Fig. 3.23
Implementing
 $F = (AB' + A'B)(C + D')$

NOR Implementation

- NOR function is the dual of NAND function
- The NOR gate is also universal

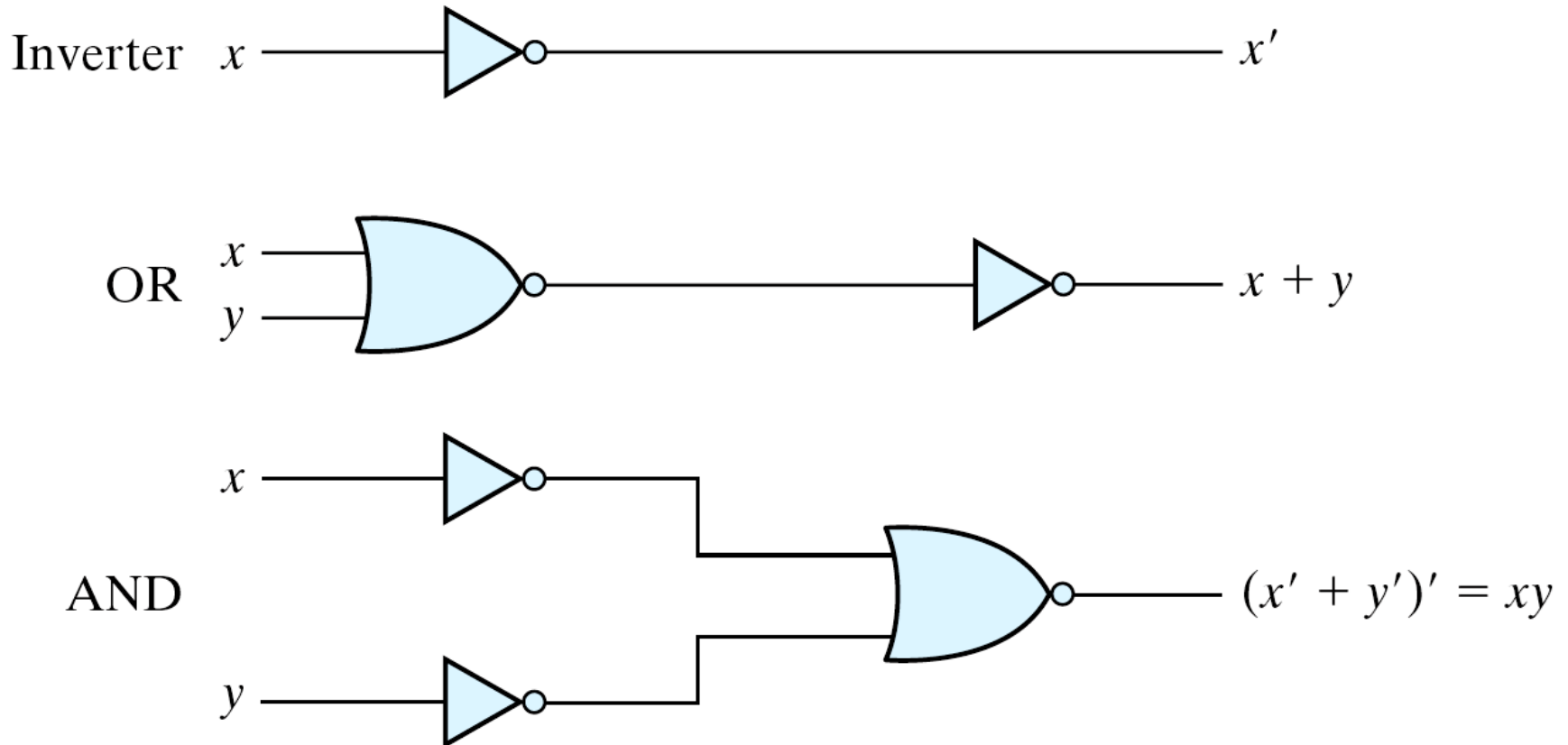
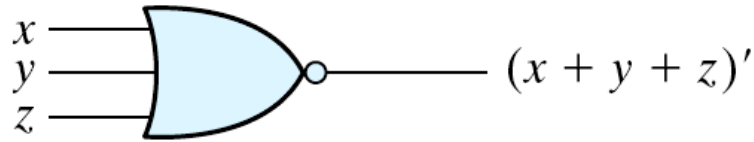
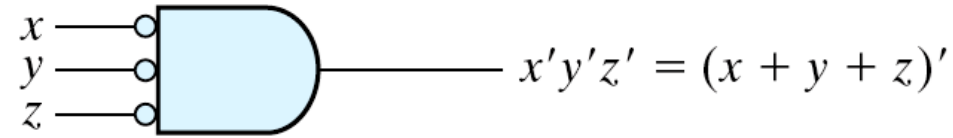


Fig. 3-24 Logic Operations with NOR Gates

Two graphic symbols for a NOR gate



(a) OR-invert



(b) Invert-AND

Fig. 3-25 Two Graphic Symbols for NOR Gate

Example: $F = (A + B)(C + D)E$

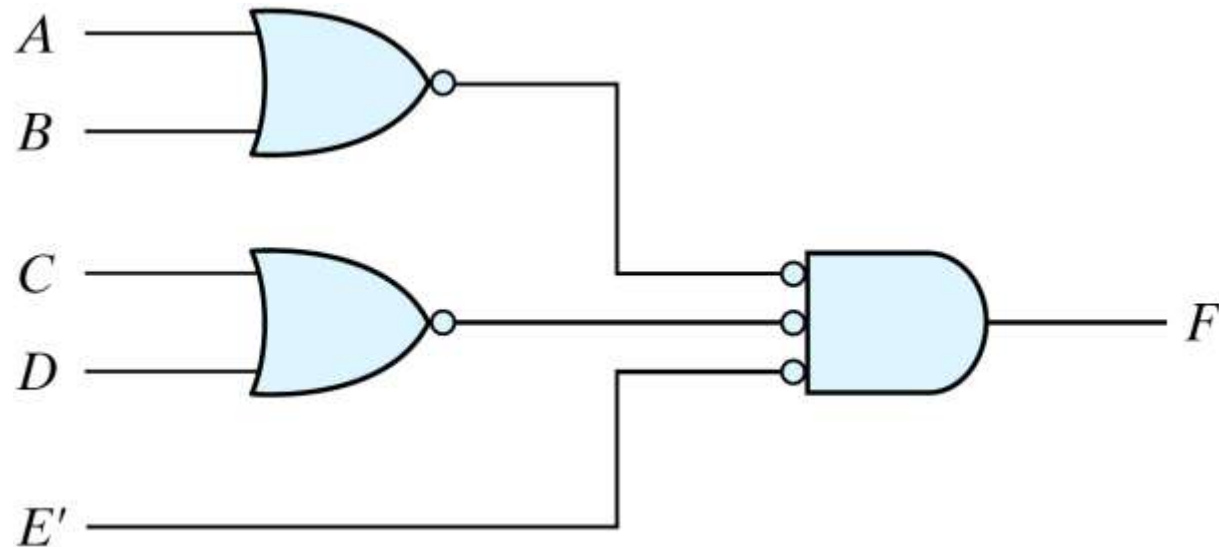


Fig. 3.26
Implementing
 $F = (A + B)(C + D)E$

Example: $F = (AB' + A'B)(C + D')$

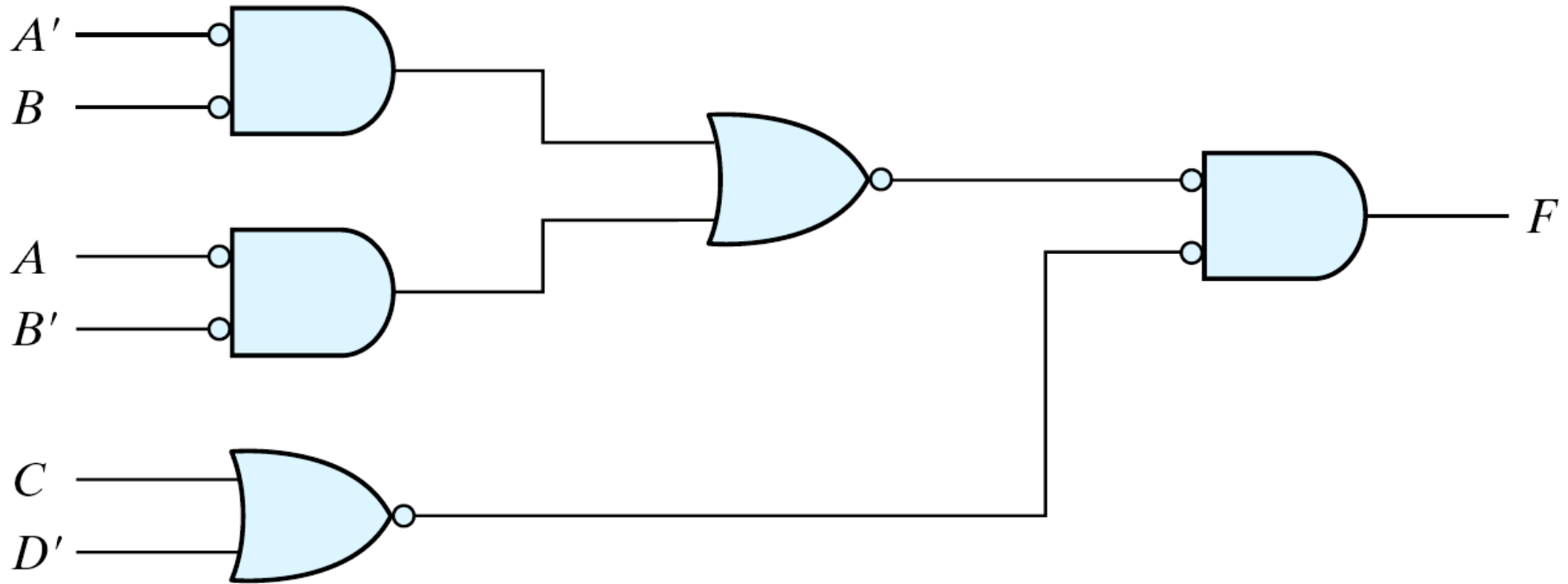


Fig. 3.27

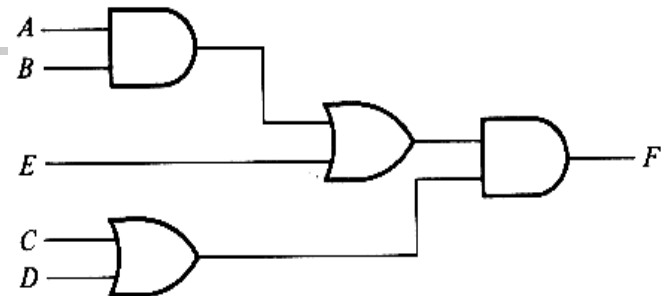
Implementing $F = (AB' + A'B)(C + D')$ with NOR gates

■ Boolean-function implementation

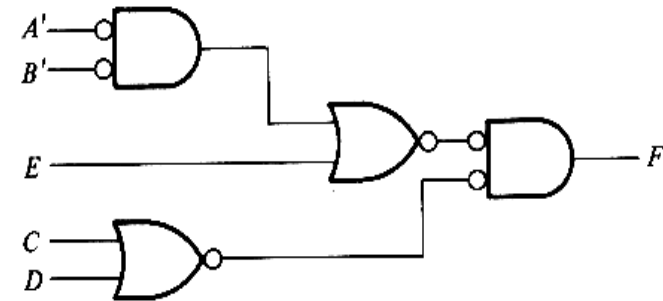
■ OR $\Rightarrow$ NOR + INV

■ AND

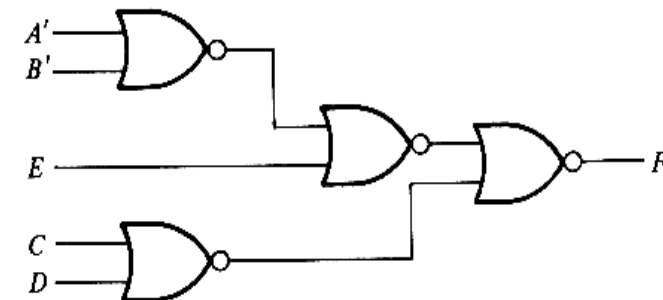
■ INV + AND = NOR



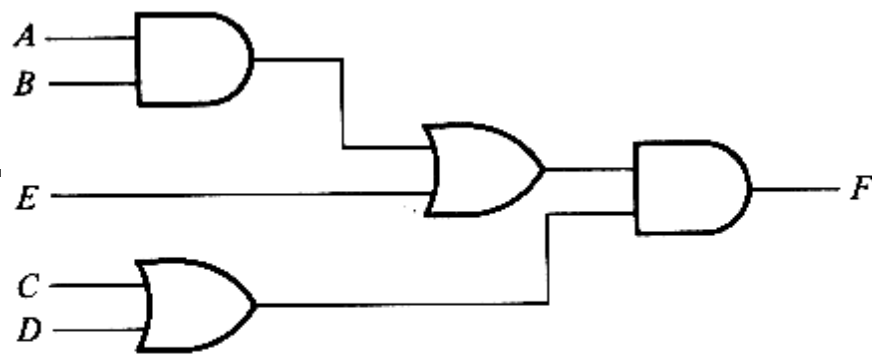
(a) AND-OR diagram



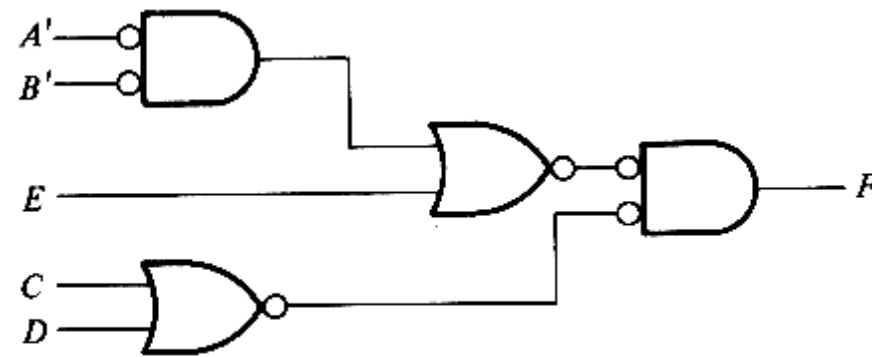
(b) NOR diagram



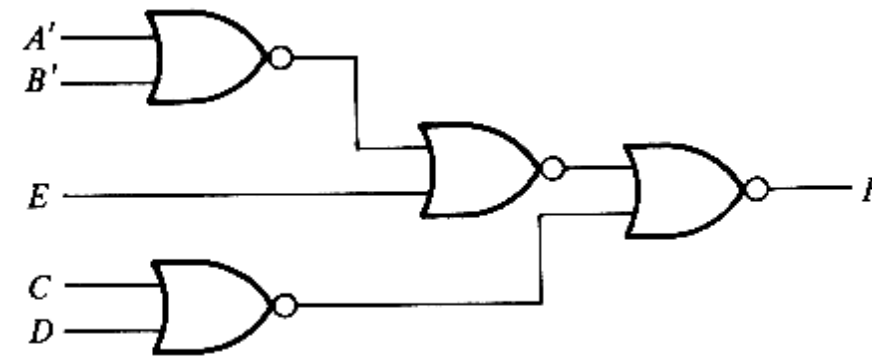
(c) Alternate NOR diagram



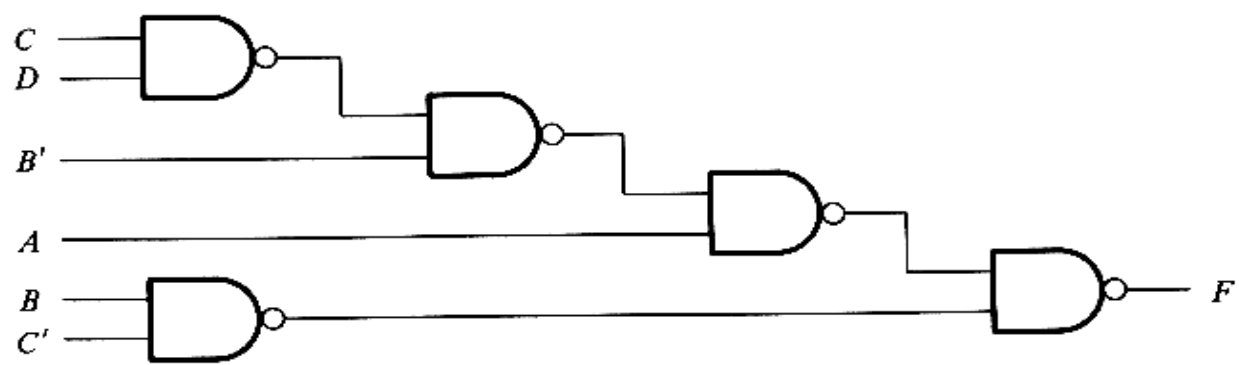
(a) AND-OR diagram



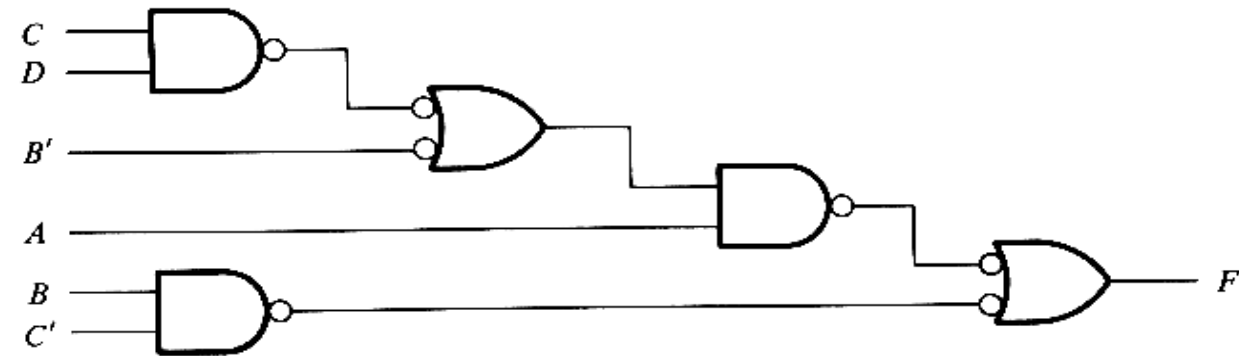
(b) NOR diagram



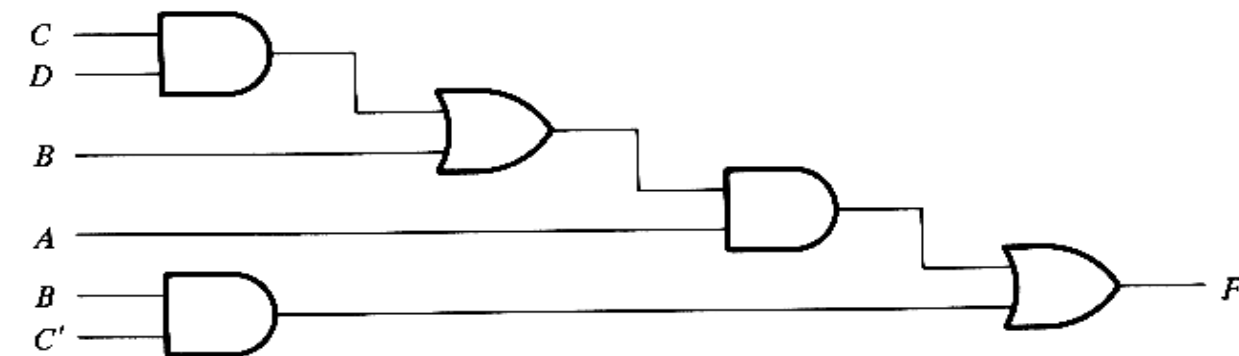
(c) Alternate NOR diagram



(a) NAND logic diagram



(b) Substitution of invert-OR symbols in alternate levels

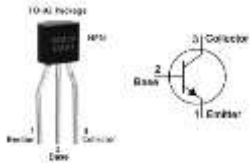


(c) AND-OR logic diagram

3-8 Other Two-level Implementations

Wired logic

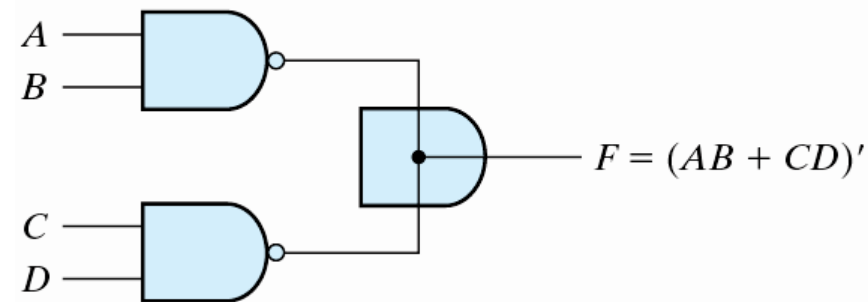
- NAND or NOR gates allow the possibility of a wire connection between the outputs of two gates to provide a specific logic function.
- open-collector TTL NAND gates: wired-AND logic
- The AND gate is drawn with the lines going through the centre of the gate to distinguish it from a conventional gate.
- A wired-logic gate does not produce a physical second-level gate, since it is just a wire connection.
- The NOR output of ECL gates: wired-OR logic
- Emitter-coupled logic (ECL) is a high-speed integrated circuit bipolar transistor logic family.



$$F = (AB)' \cdot (CD)' = (AB + CD)' = (A' + B')(C' + D')$$

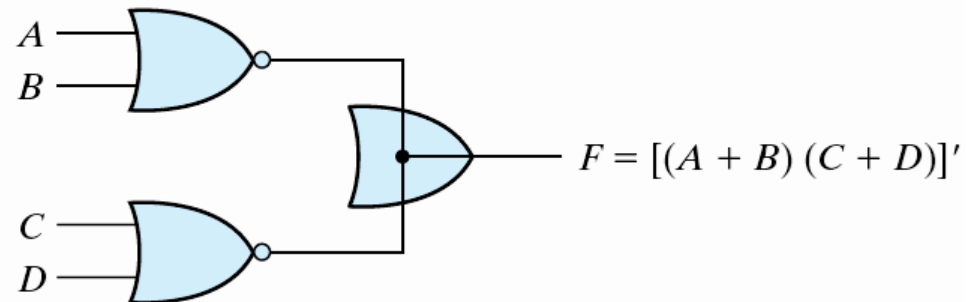
$$F = (A + B)' + (C + D)' = [(A + B)(C + D)]'$$

AND-OR-INVERT function
OR-AND-INVERT function



(a) Wired-AND in open-collector
TTL NAND gates.

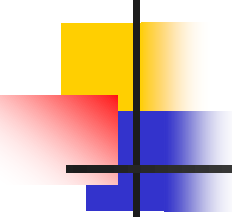
(AND-OR-INVERT)



(b) Wired-OR in ECL gates

(OR-AND-INVERT)

Fig. 3-28 Wired Logic



Nondegenerate Forms

- 16 possible combinations of two-level forms
 - eight of them: degenerate forms = a single operation
 - Eight of these combinations are said to be *degenerate* forms because they degenerate to a single operation.
 - The eight nondegenerate forms
 - AND-OR, OR-AND, NAND-NAND, NOR-NOR, NOR-OR, NAND-AND, OR-NAND, AND-NOR
 - AND-OR and NAND-NAND = sum of products
 - OR-AND and NOR-NOR = product of sums
 - **NOR-OR**, NAND-AND, **OR-NAND**, AND-NOR = ?

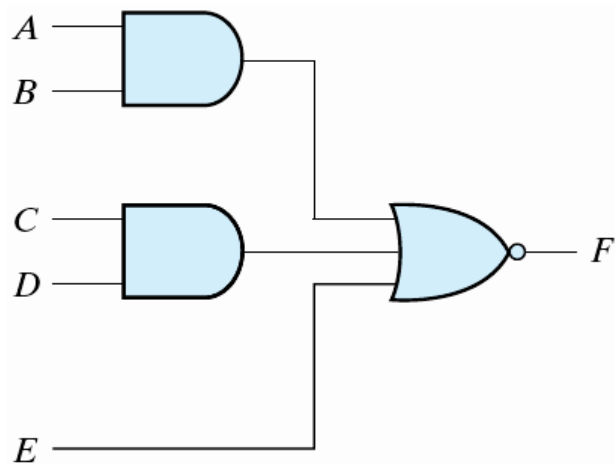
AND-OR-Invert Implementation

- AND-OR-INVERT (AOI) Implementation

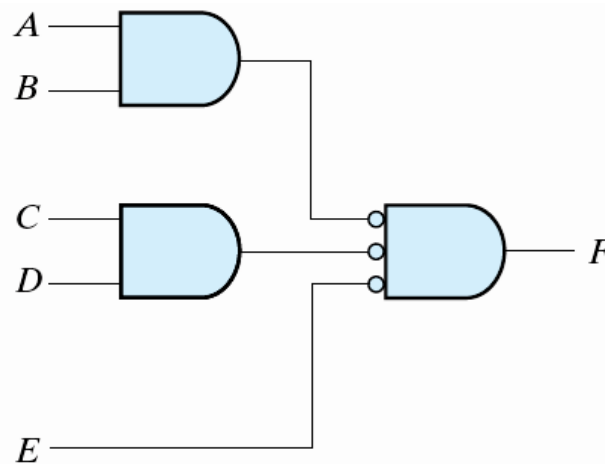
- $\text{NAND-AND} = \text{AND-NOR} = \text{AOI}$

- $F = (AB + CD + E)'$

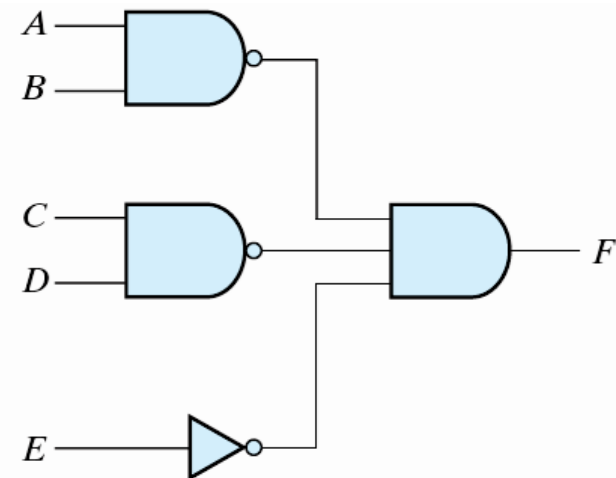
- $F' = AB + CD + E$ (sum of products)



(a) AND-NOR



(b) AND-NOR

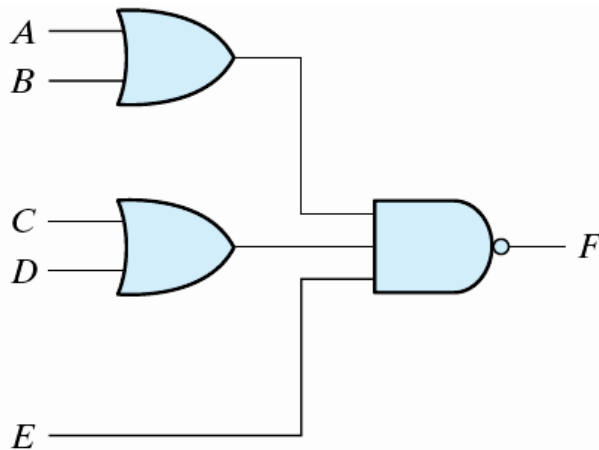


(c) NAND-AND

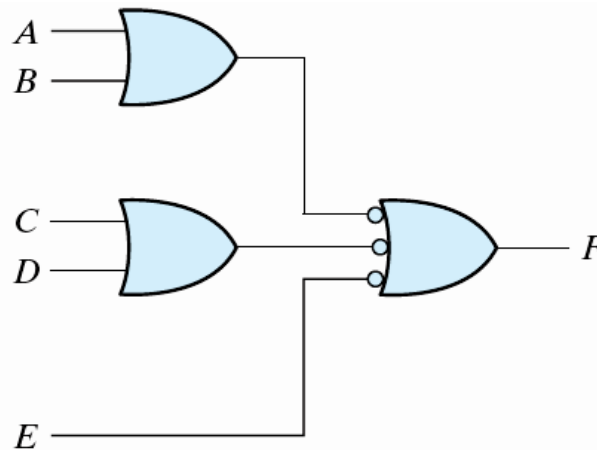
- simplify F' in sum of products

■ OR-AND-INVERT (OAI) Implementation

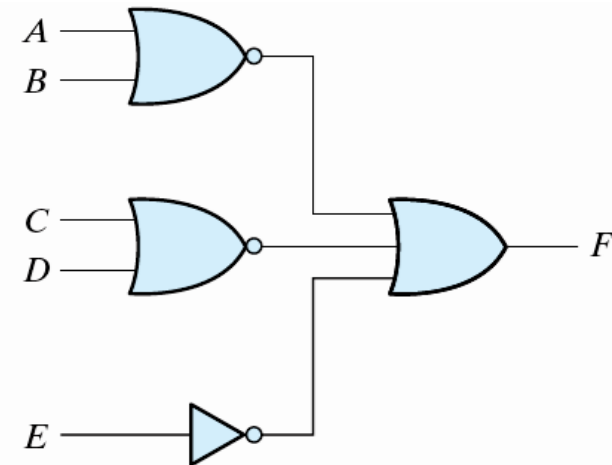
- OR-NAND = NOR-OR = OAI
- $F = ((A+B)(C+D)E)'$
- $F' = (A+B)(C+D)E$ (product of sums)



(a) OR-NAND



(b) OR-NAND



(c) NOR-OR

- simplified F' in products of sum

Tabular Summary and Examples

■ Example 3-11

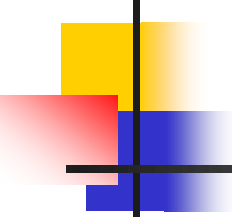
- $F' = x'y + xy' + z$ (F': sum of products)
- $F = (x'y + xy' + z)'$ (F: AOI implementation)

- $F = x'y'z' + xyz'$ (F: sum of products)
- $F' = (x+y+z)(x'+y'+z)$ (F': product of sums)
- $F = ((x+y+z)(x'+y'+z))'$ (F: OAI)

		y			
		yz			
x	0	00	01	11	10
		m_0	m_1	m_3	m_2
$x'y'z'$	0	1	0	0	0
x	1	m_4	m_5	m_7	m_6
		0	0	0	1
		z			

$F = x'y'z' + xyz'$
 $F' = x'y + xy' + z$

(a) Map simplification in sum of products

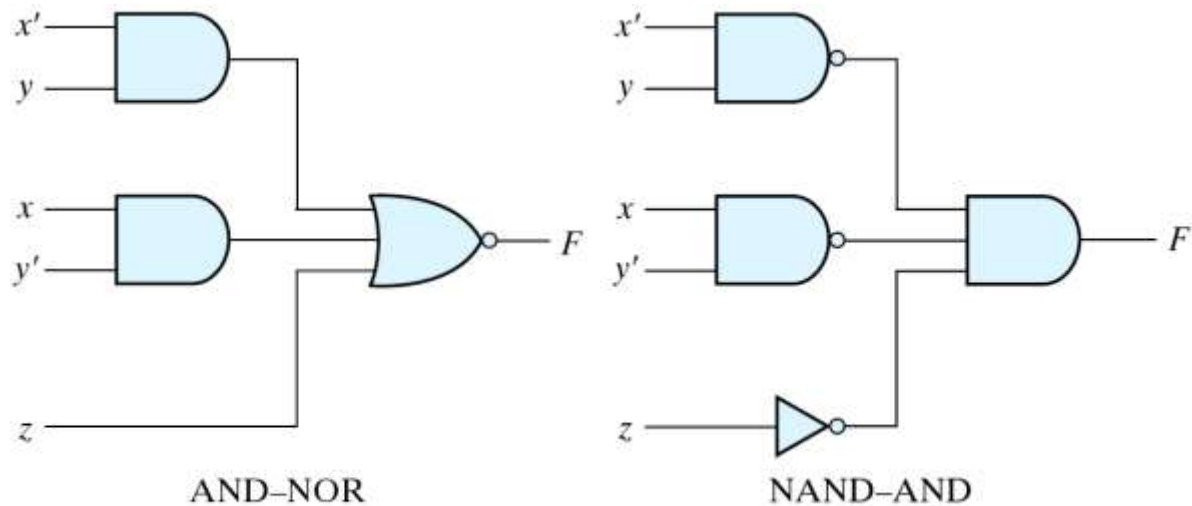


Tabular Summary and Examples

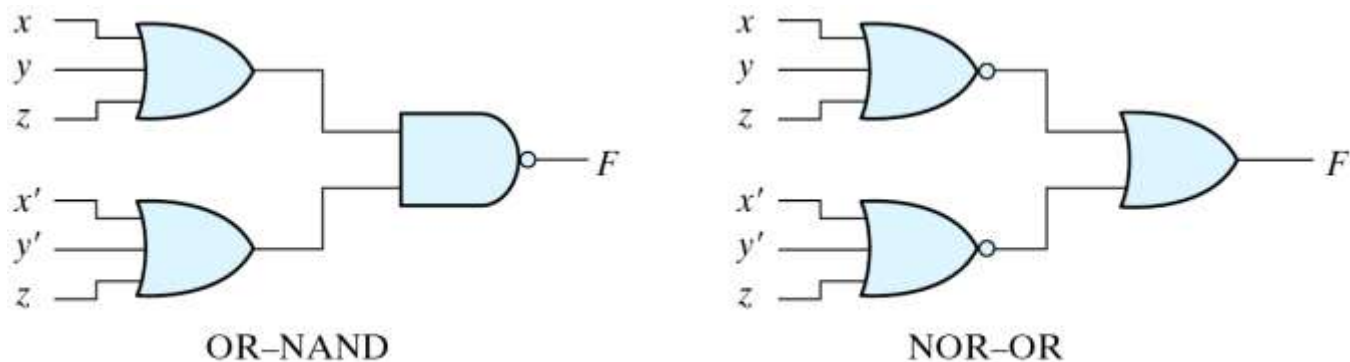
Table 3.3
Implementation with Other Two-Level Forms

Equivalent Nondegenerate Form		Implements the Function	Simplify F' into	To Get an Output of
(a)	(b)*			
AND–NOR	NAND–AND	AND–OR–INVERT	Sum-of-products form by combining 0's in the map.	F
OR–NAND	NOR–OR	OR–AND–INVERT	Product-of-sums form by combining 1's in the map and then complementing.	F

*Form (b) requires an inverter for a single literal term.

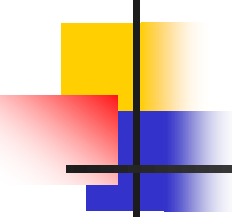


$$(b) F = (x'y + xy' + z)'$$



$$(c) F = [(x + y + z)(x' + y' + z)]'$$

Fig. 3-31 Other Two-level Implementations

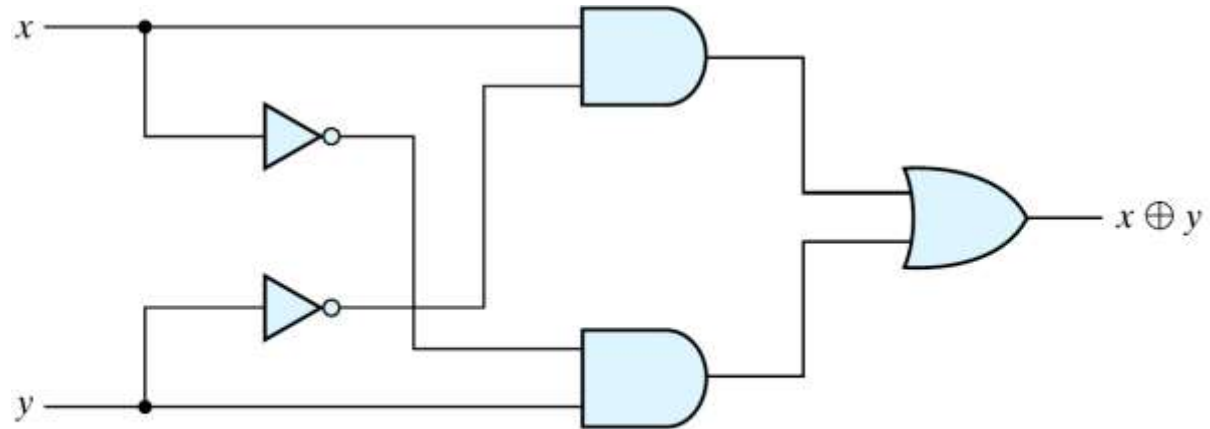


3-9 Exclusive-OR Function

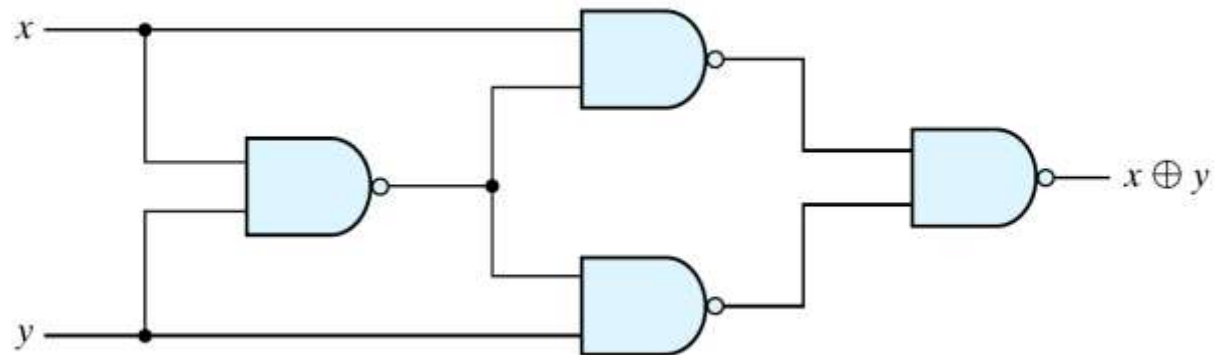
- Exclusive-OR (XOR)
 - $x \oplus y = xy' + x'y$
- Exclusive-NOR (XNOR)
 - $(x \oplus y)' = xy + x'y'$
- Some identities
 - $x \oplus 0 = x$
 - $x \oplus 1 = x'$
 - $x \oplus x = 0$
 - $x \oplus x' = 1$
 - $x \oplus y' = (x \oplus y)'$
 - $x' \oplus y = (x \oplus y)'$
- Commutative and associative
 - $A \oplus B = B \oplus A$
 - $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

■ Implementations

■ $(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$



(a) With AND-OR-NOT gates



(b) With NAND gates

Odd function

- $$A \oplus B \oplus C = (AB' + A'B)C' + (AB + A'B')C$$

$$= AB'C' + A'BC' + ABC + A'B'C$$

$$= \Sigma(1, 2, 4, 7)$$
- an odd number of 1's

		B			
		00	01	11	10
A	0	m_0	m_1 1	m_3	m_2 1
	1	m_4 1	m_5	m_7 1	m_6

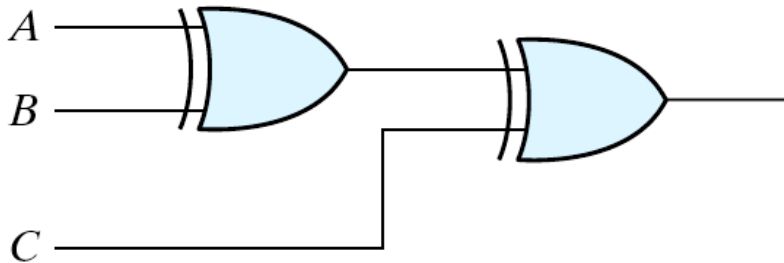
(a) Odd function $F = A \oplus B \oplus C$

		B			
		00	01	11	10
A	0	m_0 1	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7	m_6 1

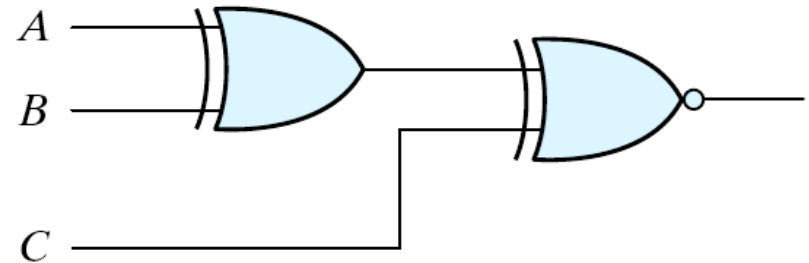
(b) Even function $F = (A \oplus B \oplus C)'$

Fig. 3-33 Map for a Three-variable Exclusive-OR Function

- Logic diagram of odd and even functions



(a) 3-input odd function



(b) 3-input even function

Fig. 3-34 Logic Diagram of Odd and Even Functions

■ Four-variable Exclusive-OR function

$$\begin{aligned} \blacksquare A \oplus B \oplus C \oplus D &= (AB' + A'B) \oplus (CD' + C'D) \\ &= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \end{aligned}$$

AB \ CD		C			
		00	01	11	10
A {	00	m_0	m_1 1	m_3	m_2 1
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12}	m_{13} 1	m_{15}	m_{14} 1
	10	m_8 1	m_9	m_{11} 1	m_{10}
		D			

} B

(a) Odd function $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A {	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1
		D			

} B

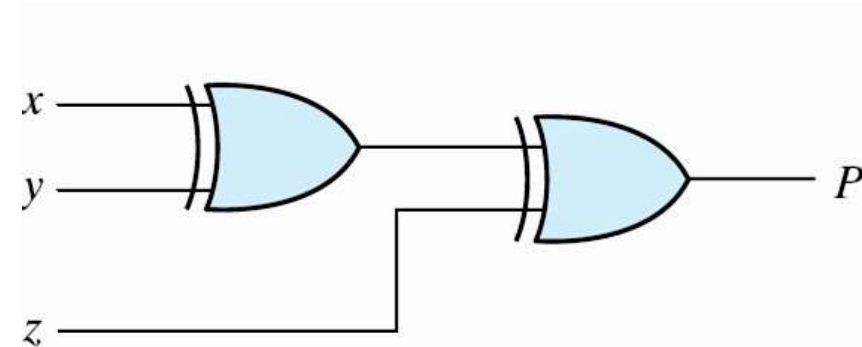
(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

Fig. 3-35 Map for a Four-variable Exclusive-OR Function

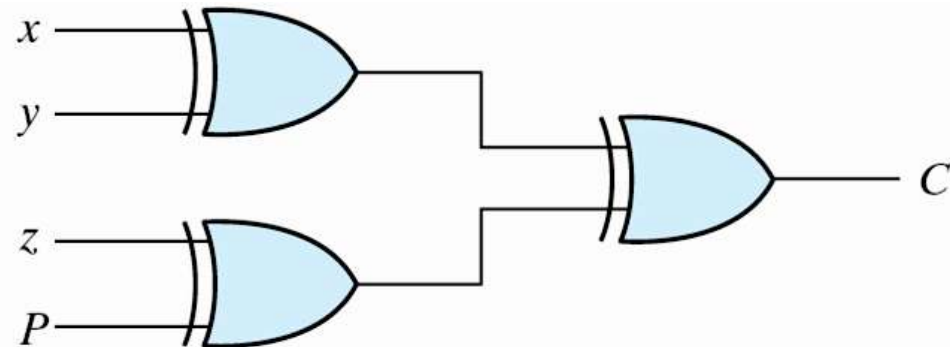
Parity Generation and Checking

■ Parity Generation and Checking

- a parity bit: $P = x \oplus y \oplus z$
- parity check: $C = x \oplus y \oplus z \oplus P$
 - $C=1$: an odd number of data bit error
 - $C=0$: correct or an even # of data bit error

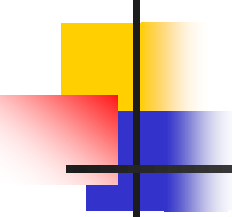


(a) 3-bit even parity generator



(b) 4-bit even parity checker

Fig. 3-36 Logic Diagram of a Parity Generator and Checker



Parity Generation and Checking

Table 3.4

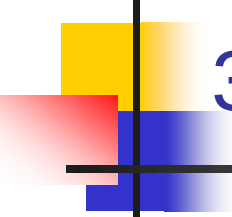
Even-Parity-Generator Truth Table

Three-Bit Message			Parity Bit
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Parity Generation and Checking

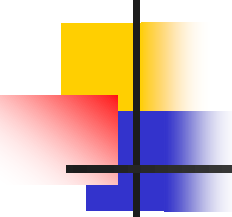
Table 3.5
Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>	<i>C</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

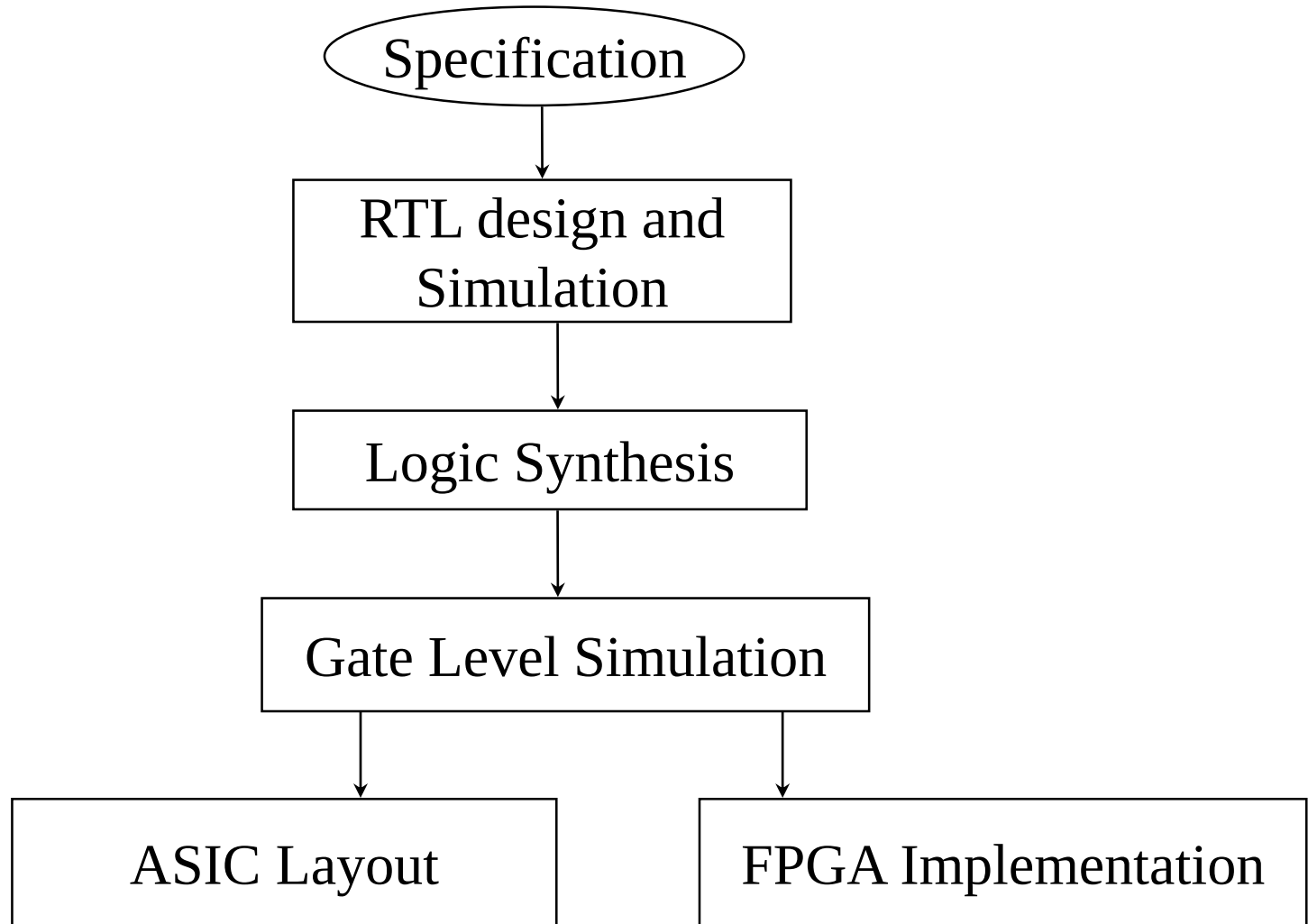


3.10 Hardware Description Language (HDL)

- Describe the design of digital systems in a textual form
 - hardware structure
 - function/behavior
 - Timing
- VHDL and Verilog HDL



A Top-Down Design Flow



Module Declaration

- Examples of keywords:
module, **end-module**, **input**, **output**, **wire**, **and**, **or**,
and **not**.

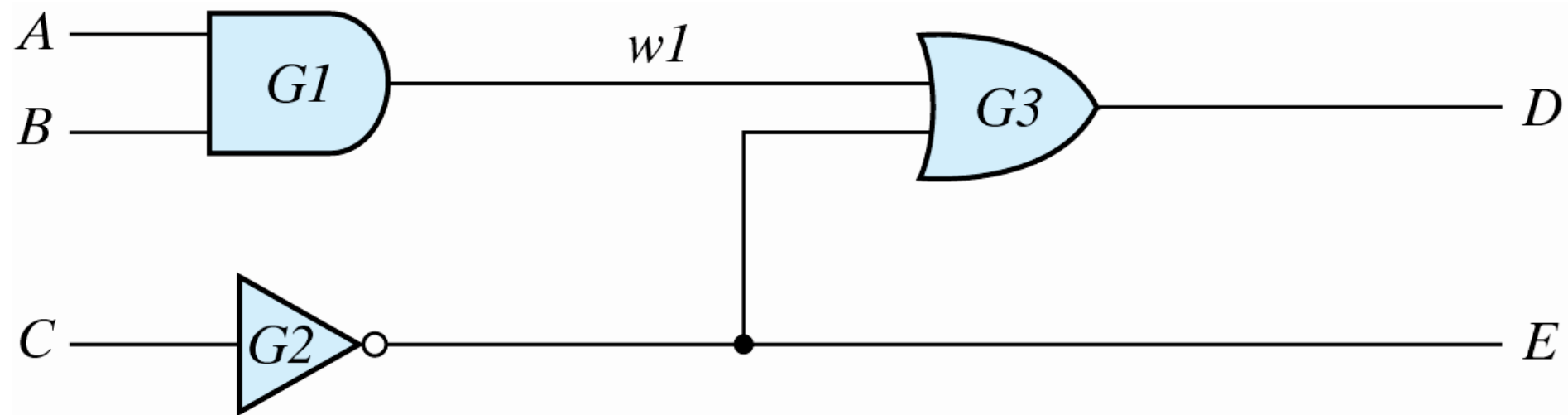
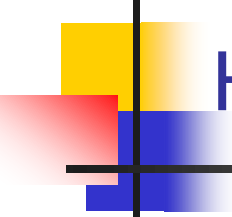


Fig. 3-37
Circuit to demonstrate an HDL



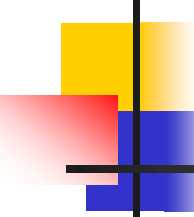
HDL Example 3.1

- HDL description for circuit shown in Fig. 3.37

HDL Example 3.1 (Combinational logic modeled with primitives)

// Verilog model of circuit of Figure 3.37. IEEE 1364–1995 Syntax

```
module Simple_Circuit (A, B, C, D, E);  
  output      D, E;  
  input       A, B, C;  
  wire        w1;  
  
  and         G1 (w1, A, B); // Optional gate instance name  
  not         G2 (E, C);  
  or          G3 (D, w1, E);  
endmodule
```



Gate Displays

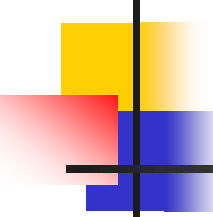
Example: timescale directive

‘timescale 1 ns/100ps

Table 3.6

Output of Gates after Delay

		Input	Output
		<i>ABC</i>	<i>E w1 D</i>
Time Units	(ns)		
Initial	—	0 0 0	1 0 1
Change	—	1 1 1	1 0 1
	10	1 1 1	0 0 1
	20	1 1 1	0 0 1
	30	1 1 1	0 1 0
	40	1 1 1	0 1 0
	50	1 1 1	0 1 1



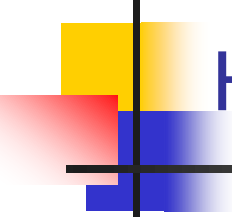
HDL Example 3.2

- Gate-level description with propagation delays for circuit shown in Fig. 3.37

HDL Example 3.2 (Gate-level model with propagation delays)

// Verilog model of simple circuit with propagation delay

```
module Simple_Circuit_prop_delay (A, B, C, D, E);  
  output D, E;  
  input   A, B, C;  
  wire    w1;  
  
  and          #(30) G1 (w1, A, B);  
  not          #(10) G2 (E, C);  
  or           #(20) G3 (D, w1, E);  
endmodule
```



HDL Example 3.3

- Test bench for simulating the circuit with delay

HDL Example 3.3

```
// Test bench for Simple_Circuit_prop_delay
```

```
module t_Simple_Circuit_prop_delay;
```

```
wire      D, E;
```

```
reg       A, B, C;
```

```
Simple_Circuit_prop_delay M1 (A, B, C, D, E); // Instance name required
```

```
initial
```

```
begin
```

```
    A = 1'b0; B = 1'b0; C = 1'b0;
```

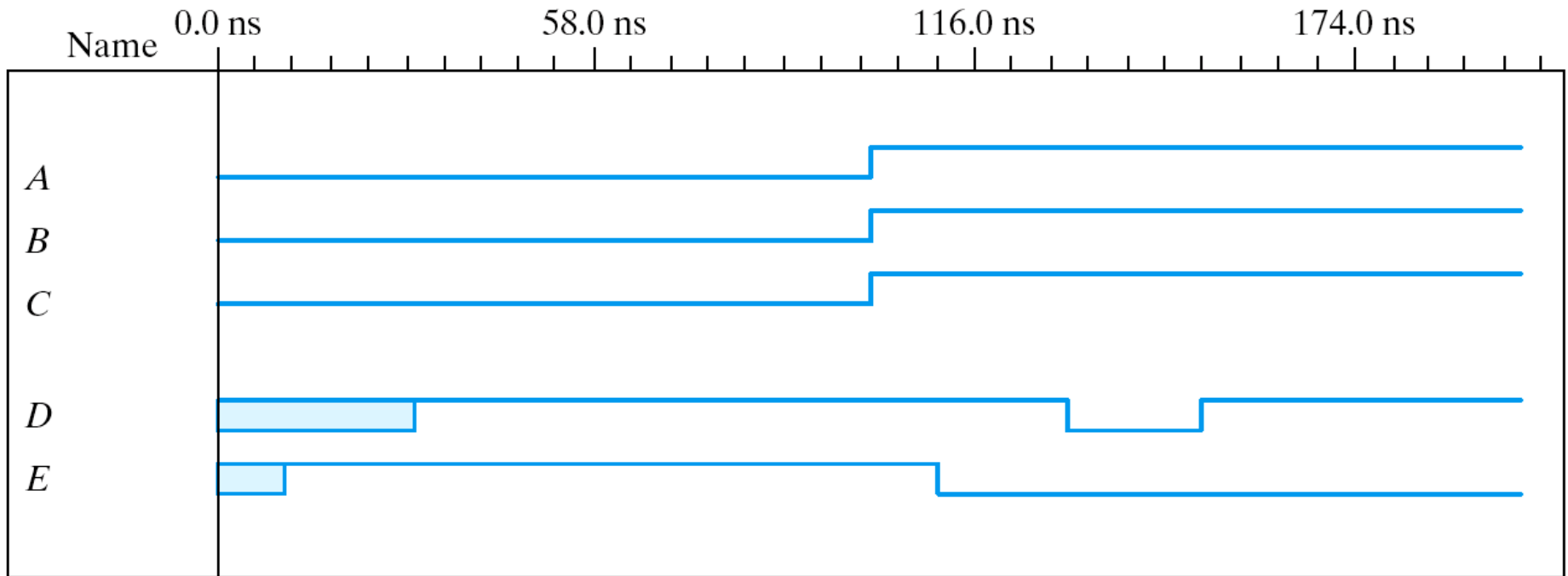
```
    #100 A = 1'b1; B = 1'b1; C = 1'b1;
```

```
end
```

```
initial #200 $finish;
```

```
endmodule
```

Simulation output for HDL Example 3.3



Boolean Expression

- ♣ Boolean expression for the circuit of Fig. 3.37

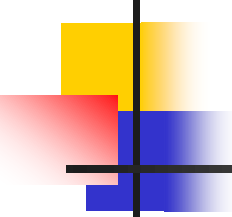
```
assign D = (A & B)|~C;
```

- ♣ Boolean expression:

$$E = A + BC + B'D$$
$$F = B'C + BC'D'$$



HDL Example 3.4



HDL Example 3.4

HDL Example 3.4 (Combinational logic modeled with Boolean equations)

// Verilog model: Circuit with Boolean expressions

module Circuit_Boolean_CA (E, F, A, B, C, D);

output E, F;

input A, B, C, D;

assign E = A | (B & C) | (~B & D);

assign F = (~B & C) | (B & ~C & ~D);

endmodule

User-Defined Primitives

♣ General rules:

- It is declared with the keyword **primitive**, followed by a name and port list.
- There can be only one output, and it must be listed first in the port list and declared with keyword **output**.
- There can be any number of inputs. The order in which they are listed in the **input** declaration must conform to the order in which they are given values in the table that follows.
- The truth table is enclosed within the keywords **table** and **endtable**.
- The values of the inputs are listed in order, ending with a colon (:). The output is always the last entry in a row and is followed by a semicolon (;).
- The declaration of a UDP ends with the keyword **endprimitive**.

♣ Declaration:

```
Circuit_with_UDP_02467 (E, F, A, B, C, D);
```



Implementing the hardware in Fig. 3.39

HDL Example 3.5

HDL Example 3.5

// Verilog model: User-defined Primitive

primitive UDP_02467 (D, A, B, C);

output D;

input A, B, C;

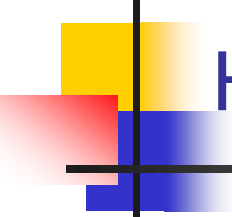
// Truth table for $D = f(A, B, C) = \Sigma(0, 2, 4, 6, 7)$;

table

//	A	B	C	:	D	// Column header comment
	0	0	0	:	1;	
	0	0	1	:	0;	
	0	1	0	:	1;	
	0	1	1	:	0;	
	1	0	0	:	1;	
	1	0	1	:	0;	
	1	1	0	:	1;	
	1	1	1	:	1;	

endtable

endprimitive



HDL Example 3.5 (Continued)

```
// Instantiate primitive
```

```
// Verilog model: Circuit instantiation of Circuit_UDP_02467
```

```
module Circuit_with_UDP_02467 (e, f, a, b, c, d);
```

```
  output      e, f;
```

```
  input       a, b, c, d;
```

```
  UDP_02467    (e, a, b, c);
```

```
  and         (f, e, d);           // Option gate instance name omitted
```

```
endmodule
```

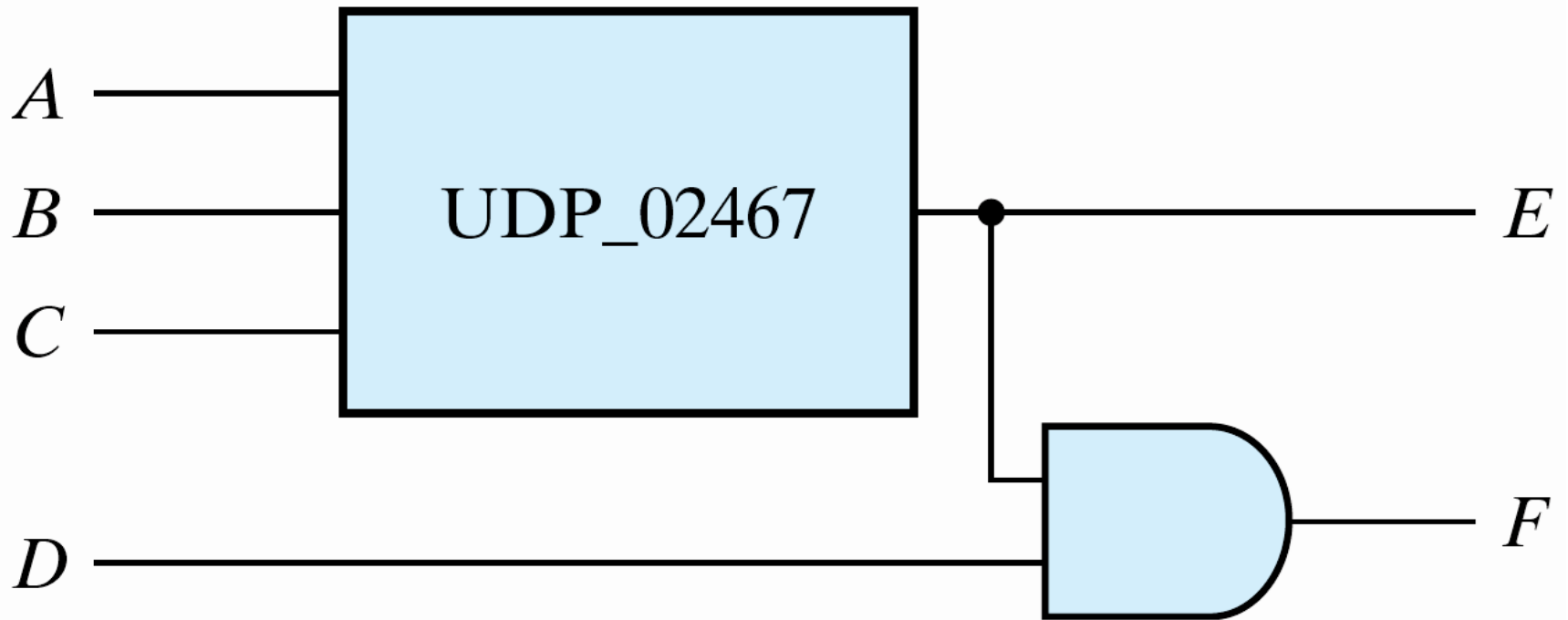


Fig. 3.39
Schematic for circuit *with_UDP_02467*